

Contents

QSPI/SPI PSRAM Controller — Documentation Index	6
Document Structure	6
Quick Reference	6
Module Hierarchy	6
Clock Domains at a Glance	7
Register Map Summary	7
01 — Overview	8
1.1 Introduction	8
1.2 Key Features	8
1.3 Top-Level Parameters	9
1.4 Sub-System Summary	9
Module Hierarchy	10
1.5 AXI Controller Detail	10
1.6 CSR Block Detail	11
1.7 QSPI FSM Detail	11
02 — Port Description	13
2.1 Clock and Reset	13
2.2 AXI4 Slave Interface	13
Write Address Channel (AW)	13
Write Data Channel (W)	14
Write Response Channel (B)	14
Read Address Channel (AR)	14
Read Data Channel (R)	15
2.3 APB Slave Interface	15
2.4 QSPI Pad Interface	16
2.5 Port Connectivity Notes	16
03 — Clocks and Resets	17
3.1 Clock Domain Overview	17
3.2 SCLK and QSPI Output Clock	18

3.3 Reset Strategy	18
3.4 Clock Frequency Constraints	19
3.5 CDC Summary	19
04 — Clock Domain Crossing (CDC)	20
4.1 ACLK ↔ SCLK: Asynchronous FIFO (Toggle-Vector Scheme)	20
4.1.1 FIFO Instances	20
4.1.2 Toggle-Vector Occupancy Scheme	20
4.1.3 Required STA Constraints	21
4.2 PCLK ↔ SCLK: 4-Phase Handshake (CSR)	22
4.2.1 Mechanism Overview	22
4.2.2 Signal Flow	23
4.2.3 Key Signals in m_vlsi_qspi_csr	23
4.2.4 Latency	24
4.2.5 Required STA Constraints	24
4.3 CDC Waiver Guidance	24
05 — Configuration & Status Registers (CSR)	26
Register Map	26
0x00 — ctrl (Control)	27
0x04 — wd (Width Descriptor)	28
0x08 — read (Read Command)	28
0x0C — write (Write Command)	29
0x10 — wr_dummy (Write Dummy Count)	29
0x14 — rd_dummy (Read Dummy Count)	30
0x18 — mode_status (Mode Status)	30
0x1C — independent (Independent Command)	31
Register Access Sequence (req/ack Pattern)	31
06 — Functional Description	32
6.1 QSPI FSM	32
6.1.1 State Summary	33
6.1.2 State Transitions	33
6.1.3 Phase Details	33

6.2 Arbitration	34
6.2.1 Arbitration Logic	34
6.2.2 Request Conditions	34
6.3 Write Flow	34
6.4 Read Flow	35
6.5 Independent Command Flow	35
6.6 XIP Mode	36
6.7 auto_data_wd Mode	36
6.8 Write Response Error (SLVERR)	36
07 — Software Integration Guide	37
7.1 Initialization Sequence	37
7.1.1 Register Configuration Order	37
7.2 Normal AXI Read/Write	38
7.3 Changing Command Opcodes at Run Time	38
7.4 Independent (Command-Only) Transaction	38
7.5 Switching from SPI to QSPI Mode	39
7.6 Switching from QSPI to SPI Mode	39
7.7 APB Access Latency Notes	40
7.8 Polling vs. Interrupt-Driven Operation	40
7.9 Bare-Metal Driver Pseudocode	40
08 — STA Constraints Reference	42
8.1 ACLK ↔ SCLK (Async FIFO Toggle-Vector Synchronizers)	42
8.2 PCLK ↔ SCLK (CSR 4-Phase Handshake Synchronizers)	42
8.3 False Path Declarations	43
8.4 Reset Path Constraints	43
8.5 Synchronizer Cell Attributes	43
8.6 Constraint Summary Table	44
8.7 CDC Tool Waiver Guidance	44
QSPI/SPI PSRAM Controller — Hardware Design Specification	45
I. Overview	45
II. Block Diagram	46

III. Port Description	47
3.1 Clock and Reset	47
3.2 AXI4 Slave Interface	47
Write Address Channel (AW)	47
Write Data Channel (W)	48
Write Response Channel (B)	48
Read Address Channel (AR)	48
Read Data Channel (R)	49
3.3 APB Slave Interface	49
3.4 QSPI Pad Interface	50
IV. Clocks and Resets	50
V. Clock Domain Crossing (CDC)	51
5.1 ACLK ↔ SCLK: Asynchronous FIFO (Toggle-Vector Scheme)	51
5.2 PCLK ↔ SCLK: 4-Phase Handshake (CSR)	52
VI. Configuration Registers (CSR)	52
0x00 — ctrl (Control)	53
0x04 — wd (Width Configuration)	53
0x08 — read (Read Command)	53
0x0C — write (Write Command)	54
0x10 — wr_dummy (Write Dummy Count)	54
0x14 — rd_dummy (Read Dummy Count)	54
0x18 — mode_status (Mode Status)	54
0x1C — independent (Independent Command)	55
VII. Functional Description	55
7.1 QSPI FSM States	55
7.2 Arbitration	57
7.3 Write Flow	57
7.4 Read Flow	57
7.5 Independent Command	58
7.6 XIP Mode	58
7.7 auto_data_wd	58
7.8 Write Response Error (SLVERR)	58

VIII. Software Integration Guide	59
8.1 Initialization	59
8.2 Normal AXI Read/Write	59
8.3 Changing Read/Write Commands at Run Time	59
8.4 Independent (Command-Only) Transaction	59
8.5 APB Access Latency	60
IX. References	60
X. Contact	60

QSPI/SPI PSRAM Controller — Documentation Index

IP Name: m_vlsi_qspi_top

Revision: 2.0

Date: 2026-06

Authors: Ethan, Link

Organization: VLSI Technology

Document Structure

Section	Contents
Overview	IP overview, key features, top-level parameters, block diagram
Port Description	Complete port list — AXI4, APB, QSPI, clock/reset
Clocks and Resets	Clock domains, reset strategy
Clock Domain Crossing	Clock Domain Crossing mechanisms and timing constraints
Configuration & Status Register (CSR)	Configuration & Status Registers — full bit-field reference
Functional Description	FSM states, arbitration, write/read/independent command flows
Software Integration Guide	Software integration guide — initialization, runtime sequences
STA Constraints Reference	STA constraints reference

Quick Reference

Module Hierarchy

```
m_vlsi_qspi_top
├─ m_vlsi_axi4_sram      (AXI Controller)
└─ m_vlsi_axfsm ×2      (AW and AR channel FSMs)
```

		m_vlsi_axi_bus_logic	(W / R / B channel logic, ACLK side)
		m_vlsi_axi_sclk_logic	(memory-side logic, SCLK side)
		m_vlsi_arbiter	(round-robin read/write arbiter)
		m_vlsi_async_fifo ×5	(AWFIFO, WFIFO, ARFIFO, RFIFO, BFIFO)
		m_vlsi_qspi_csr	(CSR – APB slave, 4-phase handshake CDC)
		m_vlsi_synch ×4	(2-stage synchronizers)
		m_vlsi_qspi_fsm	(QSPI transaction engine)

Clock Domains at a Glance

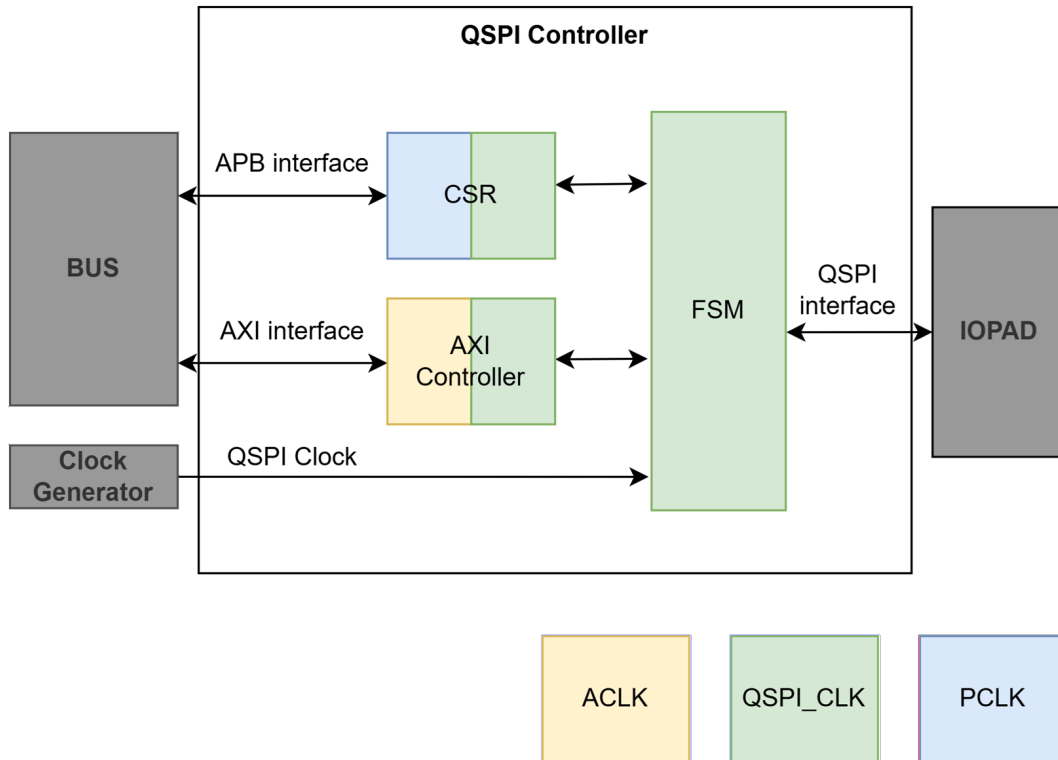
Domain	Clock	Reset	Key logic
ACLK	i_axi_clk	i_axi_rstn	AXI slave interface, async FIFO AXI side
PCLK	i_apb_clk	i_apb_rstn	APB CSR interface
SCLK	i_qspi_clk	i_qspi_rstn	QSPI FSM, CSR register file, async FIFO SCLK side

Register Map Summary

Offset	Name	Description
0x00	ctrl	Enable, auto data-width, 2-byte command mode
0x04	wd	Width descriptor (address/data width)
0x08	read	Read command opcode
0x0C	write	Write command opcode
0x10	wr_dummy	Write dummy cycle count
0x14	rd_dummy	Read dummy cycle count
0x18	mode_status	Current interface mode (read-only)
0x1C	independent	Independent (command-only) transaction

01 — Overview

1.1 Introduction



The **QSPI/SPI PSRAM Controller** (`m_vlsi_qspi_top`) is a hardware IP developed by **VLSI Technology** that bridges an AXI4 memory-mapped bus to an external PSRAM device over a QSPI (Quad SPI) or standard SPI interface.

The IP accepts AXI4 read and write transactions from a host processor, translates them into the command-address-dummy-data phase sequence required by QSPI/SPI PSRAMs, and drives the I/O pads accordingly. A separate APB slave interface provides run-time configuration of command opcodes, data widths, dummy-cycle counts, and operating mode through a set of Control and Status Registers (CSRs).

1.2 Key Features

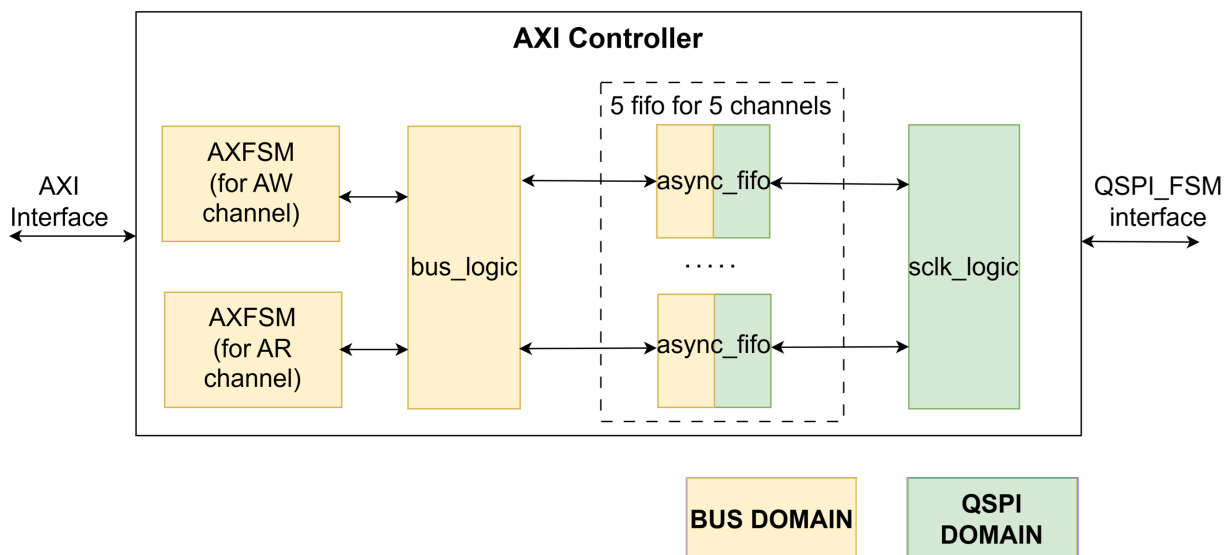
- **AXI4 full slave interface** — AW, W, B, AR, R channels with burst support (FIXED, INCR, WRAP)
- **APB slave interface** — CSR configuration and status readback
- **QSPI (4-bit I/O) and SPI (1-bit I/O)** protocol support
- **Software-controlled independent commands** — e.g., SPI→QSPI mode switching

- **Dynamic reconfiguration** — read/write command opcodes, address/data widths, and dummy cycles can be changed at run time without re-synthesis
- **XIP (Execute-In-Place)** — inferred automatically from AXI burst semantics; no dedicated control bit required
- **Round-robin arbitration** — concurrent read and write requests are served fairly in the SCLK domain
- **Three fully independent clock domains** — ACLK, PCLK, and SCLK; no assumed phase or frequency relationship between any pair

1.3 Top-Level Parameters

Parameter	Default	Description
PARAM_AXI_DATA_WD	32	AXI data bus width (bits)
PARAM_AXI_ADDR_WD	24	AXI address bus width (bits)
PARAM_AXI_ID_WD	4	AXI transaction ID width (bits)
PARAM_AXI_LEN_WD	8	AXI burst length field width (bits)
PARAM_AXI_FIFO_DEPTH	8	Depth of each asynchronous FIFO

1.4 Sub-System Summary



Sub-system	Module(s)	Clock domain	Role
AXI Controller	<code>m_vlsi_axfsm</code> ×2, <code>m_vlsi_axi_bus_logic</code> , <code>m_vlsi_axi_sclk_logic</code> , <code>m_vlsi_arbiter</code> , <code>m_vlsi_async_fifo</code> ×5	ACLK / SCLK	Accepts AXI4 transactions, crosses to SCLK, drives PSRAM interface
CSR	<code>m_vlsi_qspi_csr</code> , <code>m_vlsi_synch</code> ×4	PCLK / SCLK	APB register file with 4-phase handshake CDC
QSPI FSM	<code>m_vlsi_qspi_fsm</code>	SCLK	Core transaction engine: CMD-ADDR-DUMMY-DATA sequencing
Arbiter	<code>m_vlsi_arbiter</code>	SCLK	Round-robin read/write arbitration
Synchronizers	<code>m_vlsi_synch</code> , <code>m_vlsi_multi_synch</code>	Various	2-stage flip-flop synchronizers for FIFO CDC and CSR CDC

Module Hierarchy

```

m_vlsi_qspi_top
├─ m_vlsi_axi4_sram           (AXI Controller)
│   ├── m_vlsi_axfsm ×2      (AW and AR channel FSMs)
│   ├── m_vlsi_axi_bus_logic (W / R / B channel logic, ACLK side)
│   ├── m_vlsi_axi_sclk_logic (memory-side logic, SCLK side)
│   ├── m_vlsi_arbiter       (round-robin arbiter)
│   └─ m_vlsi_async_fifo ×5  (AWFIFO, WFIFO, ARFIFO, RFIFO, BFIFO)
├─ m_vlsi_qspi_csr           (CSR block)
│   └─ m_vlsi_synch ×4       (2-stage synchronizers for 4-phase handshake)
└─ m_vlsi_qspi_fsm           (QSPI transaction engine)

```

1.5 AXI Controller Detail

The AXI Controller (`m_vlsi_axi4_sram`) is the AXI4 slave front-end. It accepts read and write transactions from the AXI master on the `i_axi_clk` (ACLK) domain and transfers requests to the QSPI FSM on the `i_qspi_clk` (SCLK) domain through five asynchronous FIFOs.

Sub-modules:

- `m_vlsi_axfsm` (×2) — One instance per address channel (AW and AR). Each handles AXI handshaking, per-beat burst address generation, and FIFO push control.
 - `m_vlsi_axi_bus_logic` — Manages AXI data channel handshaking: receives write data (W channel), drives read data (R channel), and drives write response (B channel). Operates on the ACLK side of the async FIFOs.
 - `m_vlsi_axi_sclk_logic` — Runs in the SCLK domain. Pops requests from async FIFOs and drives the memory-side interface (address, write data, read data, control signals) toward the QSPI FSM.
 - `m_vlsi_arbiter` — Round-robin arbitration between concurrent read and write requests in the SCLK domain.
 - `m_vlsi_async_fifo` (×5) — Asynchronous FIFOs bridging ACLK and SCLK: AWFIFO, WFIFO, ARFIFO, RFIFO, BFIFO.
-

1.6 CSR Block Detail

The CSR block (`m_vlsi_qspi_csr`) provides software access to all IP configuration and status registers through the APB slave interface (PCLK domain). Register values are consumed by the QSPI FSM in the SCLK domain.

Because PCLK and SCLK are asynchronous, every APB access uses a **4-phase handshake** synchronization. This keeps PREADY deasserted until the register update has safely crossed into the SCLK domain. Four `m_vlsi_synch` 2-stage synchronizer instances implement the handshake signals.

1.7 QSPI FSM Detail

The QSPI FSM (`m_vlsi_qspi_fsm`) is the core transaction engine. It runs entirely in the SCLK domain and is responsible for:

- Receiving AXI read/write requests from the SCLK-side AXI logic
- Accepting CSR configuration: command opcodes, dummy cycle counts, SPI/QSPI mode, enable
- Generating QSPI transactions: CS assert/deassert, command phase, address phase, dummy phase, data phase
- Driving pad-side signals: `o_qspi_csn` , `io_qspi_data[3:0]` (output enable + data)
- Sampling incoming data from `io_qspi_data[3:0]` and returning it to the AXI Controller
- Handling independent command mode

- Reporting current PSRAM mode status back to the CSR (`mode_status.current`)

02 — Port Description

2.1 Clock and Reset

Port	Dir	Width	Description
<code>i_axi_clk</code>	In	1	AXI bus clock (ACLK domain)
<code>i_axi_rstn</code>	In	1	AXI active-low synchronous reset
<code>i_apb_clk</code>	In	1	APB bus clock (PCLK domain)
<code>i_apb_rstn</code>	In	1	APB active-low synchronous reset
<code>i_qspi_clk</code>	In	1	QSPI/internal logic clock (SCLK domain). Also drives <code>o_qspi_sclk</code> directly.
<code>i_qspi_rstn</code>	In	1	QSPI active-low synchronous reset

All resets are active-low and synchronous within their respective clock domain. No cross-domain reset synchronization is provided internally — the user is responsible for correct reset sequencing.

2.2 AXI4 Slave Interface

Write Address Channel (AW)

Port	Dir	Width	Description
<code>i_awvalid</code>	In	1	Write address valid
<code>o_awready</code>	Out	1	Write address ready
<code>i_awaddr</code>	In	<code>PARAM_AXI_ADDR_WD</code>	Write address
<code>i_awid</code>	In	<code>PARAM_AXI_ID_WD</code>	Write transaction ID
<code>i_awlen</code>	In	<code>PARAM_AXI_LEN_WD</code>	Burst length minus 1 (number of beats = <code>awlen + 1</code>)
<code>i_awsz</code>	In	3	Burst size: number of bytes per beat encoded as log2
<code>i_awburst</code>	In	2	

Port	Dir	Width	Description
			Burst type: 2'b00 =FIXED, 2'b01 =INCR, 2'b10 =WRAP

Write Data Channel (W)

Port	Dir	Width	Description
i_wvalid	In	1	Write data valid
o_wready	Out	1	Write data ready
i_wdata	In	PARAM_AXI_DATA_WD	Write data
i_wstrb	In	PARAM_AXI_DATA_WD/8	Write byte strobes (1 bit per byte lane)
i_wlast	In	1	Last beat of write burst

Write Response Channel (B)

Port	Dir	Width	Description
o_bvalid	Out	1	Write response valid
i_bready	In	1	Write response ready
o_bid	Out	PARAM_AXI_ID_WD	Write response ID (echoes awid)
o_bresp	Out	2	Write response code: 2'b00 =OKAY, 2'b10 =SLVERR (burst-length mismatch)

Read Address Channel (AR)

Port	Dir	Width	Description
i_arvalid	In	1	Read address valid
o_arready	Out	1	Read address ready
i_araddr	In	PARAM_AXI_ADDR_WD	Read address
i_arid	In	PARAM_AXI_ID_WD	Read transaction ID
i_arlen	In	PARAM_AXI_LEN_WD	Burst length minus 1
i_arsize	In	3	Burst size

Port	Dir	Width	Description
i_arburst	In	2	Burst type: 2'b00 =FIXED, 2'b01 =INCR, 2'b10 =WRAP

Read Data Channel (R)

Port	Dir	Width	Description
o_rvalid	Out	1	Read data valid
i_rready	In	1	Read data ready
o_rdata	Out	PARA_AXI_DATA_WD	Read data
o_rid	Out	PARA_AXI_ID_WD	Read response ID (echoes arid)
o_rlast	Out	1	Last beat of read burst
o_rresp	Out	2	Read response code: 2'b00 =OKAY

2.3 APB Slave Interface

Port	Dir	Width	Description
i_psel	In	1	APB select — selects this slave
i_penable	In	1	APB enable — second cycle of APB transfer
i_pwrite	In	1	Transfer direction: 1 =write, 0 =read
i_paddr	In	16	APB address (byte-addressed; only [4:0] used for register offset)
i_pwdata	In	32	APB write data
i_pstrb	In	4	APB write strobes (byte-enable, one bit per byte)
i_pprot	In	3	APB protection attributes
i_protect_en	In	1	Enable protection check on pprot[1] (privileged access only)
i_slverr_en	In	1	Enable SLVERR response on address or strobe violations
o_prdata	Out	32	APB read data

Port	Dir	Width	Description
<code>o_pready</code>	Out	1	APB ready — held low until CDC round-trip completes
<code>o_pslverr</code>	Out	1	APB slave error

`o_pready` is deasserted for the duration of the 4-phase CDC handshake. Minimum stall is approximately 2 SCLK cycles + 2 PCLK cycles. See [04_cdc.md](#) for details.

2.4 QSPI Pad Interface

Port	Dir	Width	Description
<code>o_qspi_sclk</code>	Out	1	QSPI clock output — direct wire: <code>assign o_qspi_sclk = i_qspi_clk</code>
<code>o_qspi_csn</code>	Out	1	QSPI chip-select (active low)
<code>io_qspi_data</code>	Inout	4	Quad SPI data lines DQ[3:0]; tri-stated during dummy and idle phases

`o_qspi_sclk` is a **combinational pass-through** — there is no clock gating. The SCLK output runs continuously whenever `i_qspi_clk` is toggling, regardless of whether a transaction is in progress.

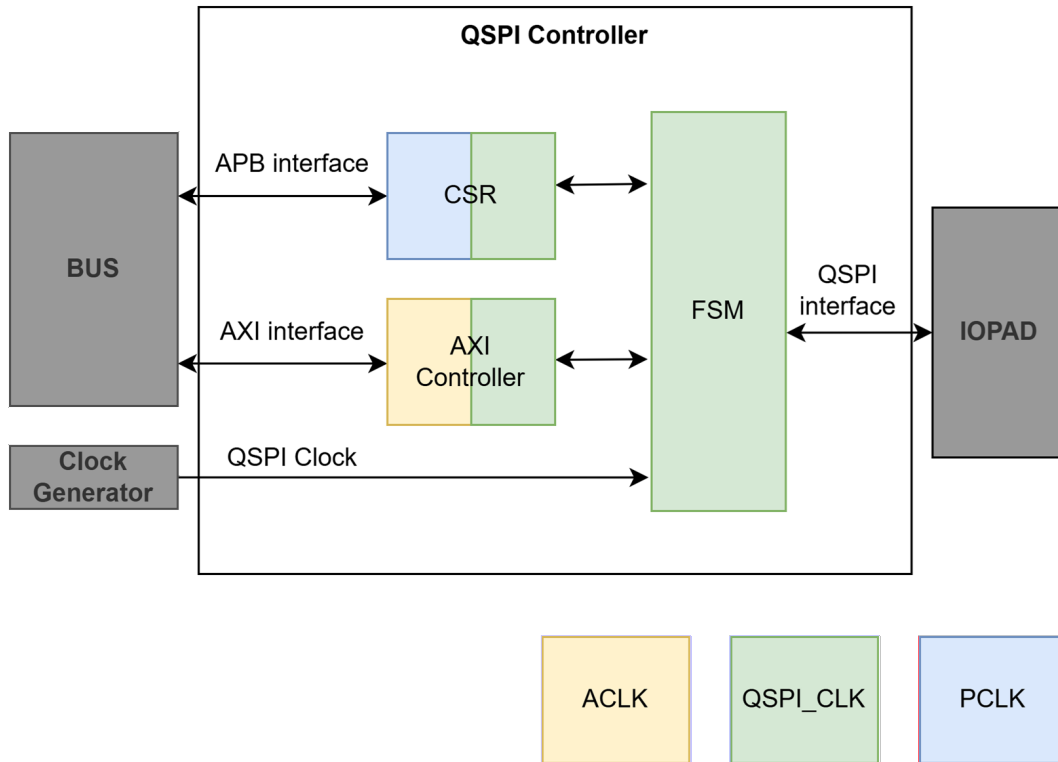
2.5 Port Connectivity Notes

- `io_qspi_data[0]` is MOSI in SPI mode; all four bits are driven in QSPI mode.
- `io_qspi_data[1]` is MISO in SPI mode; it is driven by the IP during QSPI writes.
- The output-enable signals for `io_qspi_data` are controlled internally by the FSM based on the current phase (command/address/write = driven; dummy/read = high-Z).
- The IP does not include I/O pad cells. The user must instantiate appropriate pad cells for `io_qspi_data` with separate output-enable pins in the SoC integration wrapper.

03 — Clocks and Resets

3.1 Clock Domain Overview

The IP operates across **three fully independent, asynchronous clock domains**. No phase or frequency relationship is assumed between any pair of clocks.



Domain	Clock signal	Reset signal	Description
ACLK	<code>i_axi_clk</code>	<code>i_axi_rstn</code>	AXI4 bus interface — drives AXI slave logic and the AXI-side ports of the async FIFOs (write side of AWFIFO/ WFIFO/ARFIFO; read side of RFIFO/ BFIFO)
PCLK	<code>i_apb_clk</code>	<code>i_apb_rstn</code>	APB bus interface — drives the CSR APB access logic and the PCLK side of the 4-phase handshake synchronizers
SCLK	<code>i_qspi_clk</code>	<code>i_qspi_rstn</code>	QSPI / internal logic clock — drives the QSPI FSM, CSR register file, round-robin arbiter, SCLK-side AXI logic, and the SCLK-side ports of all async FIFOs

Domain	Clock signal	Reset signal	Description
			(read side of AWFIFO/WFIFO/ARFIFO; write side of RFIFO/BFIFO)

3.2 SCLK and QSPI Output Clock

`o_qspi_sclk` is a direct wire assignment to `i_qspi_clk`:

```
assign o_qspi_sclk = i_qspi_clk;
```

There is no clock gating on the SCLK output. The PSRAM device sees a continuous clock whenever `i_qspi_clk` is toggling. The PSRAM chip-select (`o_qspi_csn`) gates transactions at the protocol level.

3.3 Reset Strategy

All three resets are **active-low** and use a **synchronous de-assert** within their respective clock domain. Each domain has an independent reset:

Reset	Associated clock	Asserted by	De-assert style
<code>i_axi_rstn</code>	<code>i_axi_clk</code>	External (SoC PMU or reset controller)	Synchronous on <code>i_axi_clk</code> rising edge
<code>i_apb_rstn</code>	<code>i_apb_clk</code>	External	Synchronous on <code>i_apb_clk</code> rising edge
<code>i_qspi_rstn</code>	<code>i_qspi_clk</code>	External	Synchronous on <code>i_qspi_clk</code> rising edge

No internal cross-domain reset synchronization is provided. The user is responsible for correct reset sequencing. The recommended order is: 1. Assert all three resets simultaneously (drive all low). 2. Wait for all clocks to be stable and locked. 3. De-assert `i_qspi_rstn` first (SCLK domain — the register file and FSM must be initialized before the other domains can issue requests). 4. De-assert `i_axi_rstn` and `i_apb_rstn` after `i_qspi_rstn` has been released for at least 2 SCLK cycles.

3.4 Clock Frequency Constraints

The IP imposes no minimum or maximum relationship between ACLK, PCLK, and SCLK frequencies. However, practical performance and latency considerations apply:

- **SCLK directly determines PSRAM transaction throughput.** Higher SCLK = higher bandwidth. The PSRAM device datasheet specifies the maximum allowable SCLK frequency.
 - **ACLK should be $\geq$ SCLK** for best AXI throughput. If $ACLK < SCLK$, the async FIFOs may starve the SCLK side.
 - **PCLK** only affects APB register access latency. The CDC round-trip for each APB access is approximately $2 \times T_{SCLK} + 2 \times T_{PCLK}$. Lower PCLK increases the APB stall duration.
-

3.5 CDC Summary

Crossing	Mechanism	Module
ACLK $\rightarrow$ SCLK (data)	Async FIFO with toggle-vector occupancy tracking	<code>m_vlsi_async_fifo</code> + <code>m_vlsi_multi_synch</code>
SCLK $\rightarrow$ ACLK (data)	Async FIFO with toggle-vector occupancy tracking	<code>m_vlsi_async_fifo</code> + <code>m_vlsi_multi_synch</code>
PCLK $\leftrightarrow$ SCLK (CSR)	4-phase handshake with 2-stage synchronizers	<code>m_vlsi_qspi_csr</code> + <code>m_vlsi_synch</code> $\times 4$

See [04_cdc.md](#) for full CDC detail and required STA constraints.

04 — Clock Domain Crossing (CDC)

The IP crosses between three clock domains. Two independent CDC mechanisms are used: **asynchronous FIFOs** for ACLK↔SCLK data paths, and a **4-phase handshake** for PCLK↔SCLK CSR accesses.

4.1 ACLK ↔ SCLK: Asynchronous FIFO (Toggle-Vector Scheme)

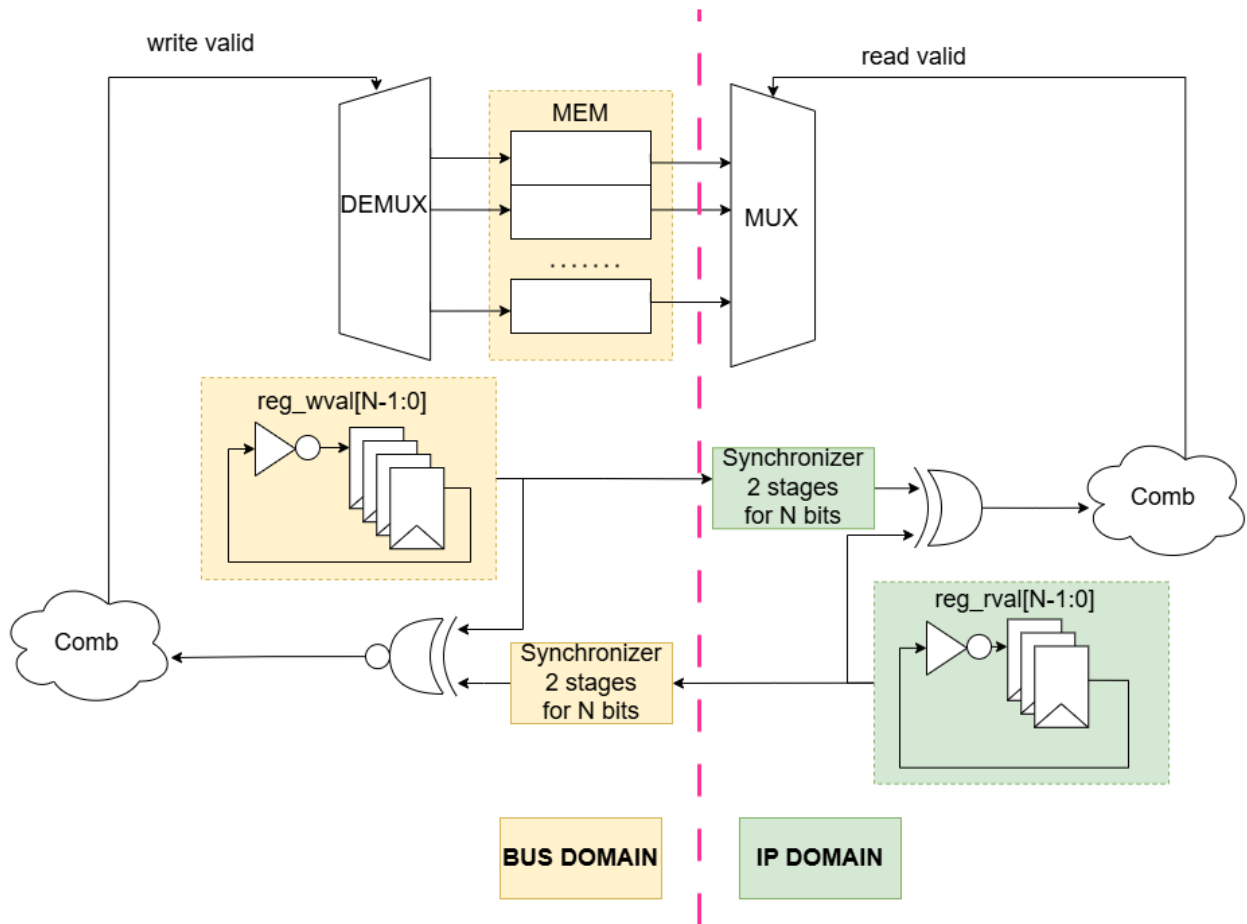
4.1.1 FIFO Instances

Five async FIFOs (`m_vlsi_async_fifo`) bridge ACLK and SCLK:

FIFO	Direction	Data contents
AWFIFO	ACLK → SCLK	{aw_last, aw_id[ID_WD-1:0], aw_addr[ADDR_WD-1:0]}
WFIFO	ACLK → SCLK	{w_last, w_data[DATA_WD-1:0]}
ARFIFO	ACLK → SCLK	{ar_last, ar_id[ID_WD-1:0], ar_addr[ADDR_WD-1:0]}
BFIFO	SCLK → ACLK	{aw_id[ID_WD-1:0], bresp[1:0]}
RFIFO	SCLK → ACLK	{rd_id[ID_WD-1:0], 2'b00, rdata[DATA_WD-1:0], rd_last}

4.1.2 Toggle-Vector Occupancy Scheme

Rather than Gray-coded binary pointers, these FIFOs use a **toggle-vector** scheme. Each side maintains a shift register of width `PARAM_DEPTH` that tracks occupancy state.



Each direction uses a `m_vlsi_multi_synch` instance — a `PARAM_DEPTH`-wide 2-stage synchronizer — to safely transfer the occupancy vector to the opposite clock domain.

4.1.3 Required STA Constraints

```
# ACLK → SCLK: FIFO write paths
set_max_delay -from [get_clocks ACLK] -to [get_clocks SCLK] \
    [expr {1.5 * $SCLK_period}]

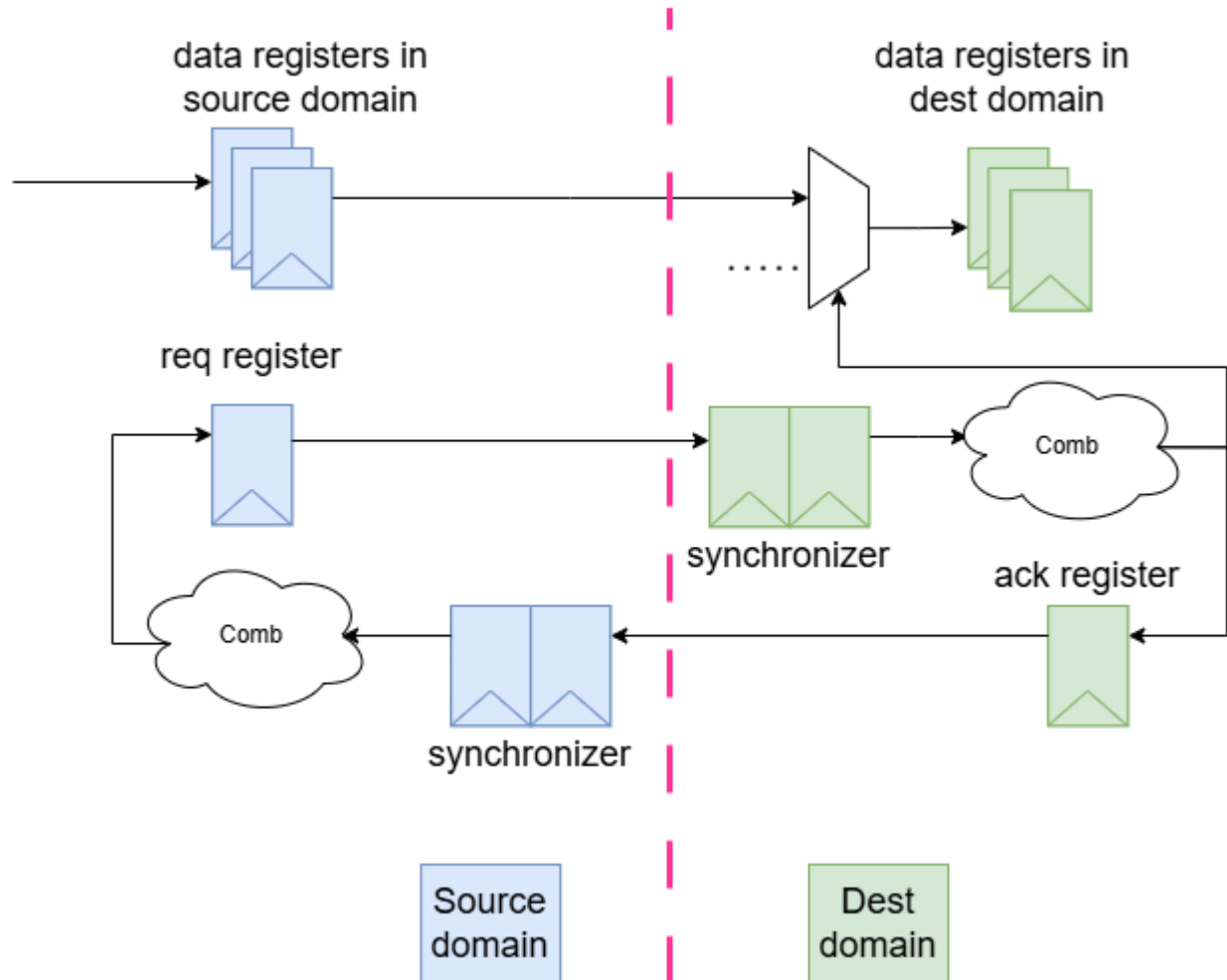
# SCLK → ACLK: FIFO read paths
set_max_delay -from [get_clocks SCLK] -to [get_clocks ACLK] \
    [expr {1.5 * $ACLK_period}]
```

Apply these constraints to all paths through `m_vlsi_multi_synch` instances inside `m_vlsi_async_fifo`.

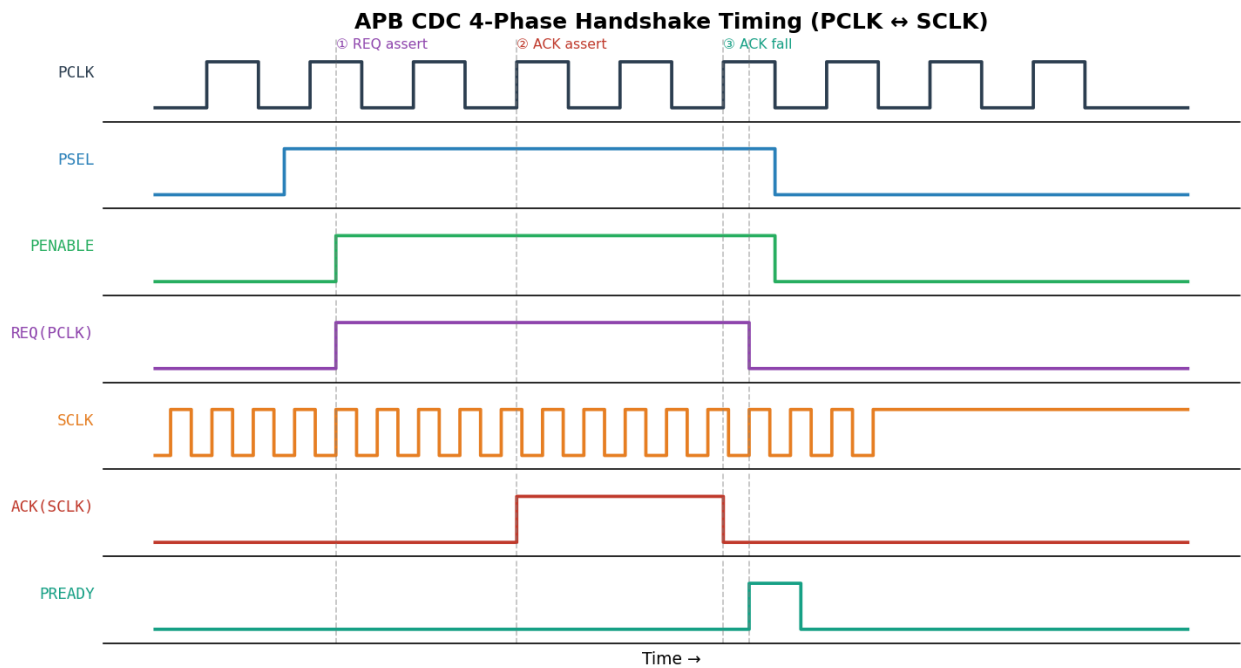
4.2 PCLK ↔ SCLK: 4-Phase Handshake (CSR)

4.2.1 Mechanism Overview

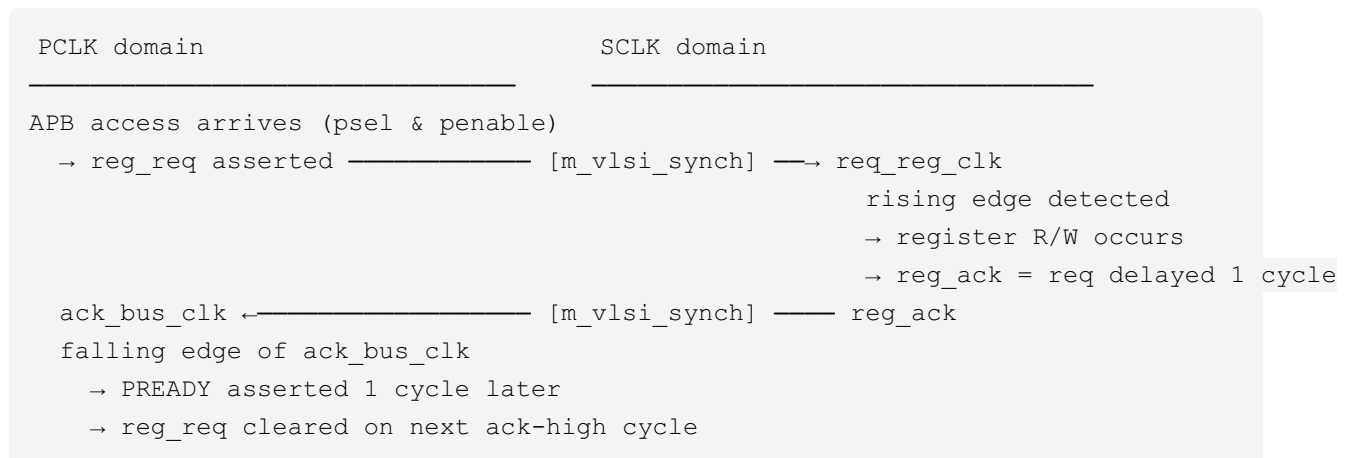
Every APB register access (read or write) crosses into the SCLK domain via a 4-phase handshake.



The APB bus is stalled (`o_pready = 0`) for the entire round-trip CDC latency.



4.2.2 Signal Flow



4.2.3 Key Signals in `m_vlsi_qspi_csr`

Signal	Domain	Description
<code>reg_read_req</code>	PCLK	Read request flop, set on APB read access
<code>reg_write_req</code>	PCLK	Write request flop, set on APB write access
<code>read_req_reg_clk</code>	SCLK	<code>reg_read_req</code> synchronized into SCLK; rising edge triggers read
<code>write_req_reg_clk</code>	SCLK	<code>reg_write_req</code> synchronized into SCLK; rising edge triggers write
<code>read_ack</code>	SCLK	

Signal	Domain	Description
		One-cycle delayed copy of <code>read_req_reg_clk</code> ; used as ACK back to PCLK
<code>write_ack</code>	SCLK	One-cycle delayed copy of <code>write_req_reg_clk</code>
<code>read_ack_bus_clk</code>	PCLK	<code>read_ack</code> synchronized back to PCLK
<code>write_ack_bus_clk</code>	PCLK	<code>write_ack</code> synchronized back to PCLK

`o_pready` asserts one PCLK cycle after the falling edge of `read_ack_bus_clk` or `write_ack_bus_clk`.

4.2.4 Latency

Minimum APB stall duration (from `penable` to `pready`):

$$T_{\text{stall}} \approx 2 \times T_{\text{SCLK}} + 2 \times T_{\text{PCLK}} + \text{register setup margins}$$

This is the synchronizer pipeline depth (2 stages each direction). At typical frequencies, this is on the order of a few to tens of nanoseconds — negligible for software-driven CSR accesses.

4.2.5 Required STA Constraints

```
# PCLK → SCLK: Write data paths
set_max_delay -from [get_clocks PCLK] -to [get_clocks SCLK] \
    [expr {1.5 * $SCLK_period}]

# SCLK → PCLK: Read data paths
set_max_delay -from [get_clocks SCLK] -to [get_clocks PCLK] \
    [expr {1.5 * $PCLK_period}]
```

Apply these constraints to all paths through `m_vlsi_synch` instances inside `m_vlsi_qspi_csr`.

4.3 CDC Waiver Guidance

If the above constraints are correctly applied and passing in STA:

- All paths through `m_vlsi_multi_synch` (async FIFO vector synchronizers) can be waived in formal CDC analysis as "synchronized by max-delay constraint."
- All paths through `m_vlsi_synch` inside the CSR block can be waived similarly.

- Any remaining CDC paths are either false paths (no data crossing) or are covered by the above mechanisms.
- The release provides CDC folder with CDC waiver file, which can be used as reference

Formal CDC tools (e.g., Synopsys VC Static CDC, Cadence JasperGold CDC) should be configured to recognize these synchronizer cells to avoid false violations.

05 — Configuration & Status Registers (CSR)

All registers are **32 bits wide** and accessed through the APB slave interface. The register file is clocked by SCLK; every APB access is synchronized via the 4-phase handshake described in [04_cdc.md](#).

Access type key:

Code	Meaning
RW	Software read/write
RO	Read-only; software writes are ignored
RWI	Read/write by software; hardware may also write (e.g., auto-clear <code>req</code> after ACK)

CSR Register Bit-Field Map



Register Map

Offset	Name	Description
0x00	ctrl	Global control

Offset	Name	Description
0x04	wd	Address/data width descriptor
0x08	read	Read command opcode
0x0C	write	Write command opcode
0x10	wr_dummy	Write dummy cycle count
0x14	rd_dummy	Read dummy cycle count
0x18	mode_status	Current interface mode (read-only)
0x1C	independent	Independent command control

0x00 — `ctrl` (Control)

Bits	Field	Access	Reset	Description
[31:3]	—	—	0	Reserved. Write as zero; read returns zero.
[2]	<code>cmd_2bytes</code>	RW	0	Command width select. <code>0</code> : 8-bit (1-byte) command. <code>1</code> : 16-bit (2-byte) command. Affects how <code>read.cmd</code> , <code>write.cmd</code> , and <code>independent.cmd</code> are interpreted.
[1]	<code>auto_data_wd</code>	RW	0	Auto data-width mode. <code>1</code> : FSM overrides <code>wd.data</code> with <code>PARAM_DATA_WD-1</code> on every transaction, ignoring the configured <code>wd.data</code> value.
[0]	<code>en</code>	RW	0	IP enable. Must be set to <code>1</code> before issuing AXI transactions. When <code>0</code> , the FSM stays in <code>S_IDLE</code> and ignores all requests.

0x04 — `wd` (Width Descriptor)

Bits	Field	Access	Reset	Description
[31]	<code>req</code>	RWI	0	Width change request. Write <code>1</code> to submit. Hardware clears this bit after the SCLK domain acknowledges the update.
[30]	<code>ack</code>	RO	0	Width change acknowledge. Hardware asserts when the new width values have been latched in the SCLK domain. Cleared automatically when <code>req</code> clears.
[29:24]	<code>addr</code>	RW	0	Address width field (6-bit value, unit: bits). Specifies the number of address bits sent in the ADDR phase of a QSPI transaction.
[23:0]	<code>data</code>	RW	0	Data width field (24-bit value, unit: bits). Specifies the number of data bits per PSRAM access. Ignored when <code>ctrl.auto_data_wd = 1</code> .

Usage: Write `addr` and `data`, then set `req = 1`. Poll `ack` until it asserts. Both `req` and `ack` clear automatically; no explicit clear is needed.

0x08 — `read` (Read Command)

Bits	Field	Access	Reset	Description
[31]	<code>req</code>	RWI	0	Read command update request. Write <code>1</code> to submit. Hardware clears after ACK.
[30]	<code>ack</code>	RO	0	Read command acknowledge. Asserts when the new command is latched in SCLK domain.
[29:16]	—	—	0	Reserved.
[15:0]	<code>cmd</code>	RW	0	Read command opcode. If <code>ctrl.cmd_2bytes = 0</code> , only bits [7:0] are used. If <code>ctrl.cmd_2bytes = 1</code> , all 16 bits are sent MSB-first as two command bytes.

Typical values:

PSRAM mode	Opcode	cmd_2bytes
SPI read	0x03	0
SPI fast read	0x0B	0
QSPI read	0xEB	0
Read ID	Device-specific	0 or 1

0x0C — `write` (Write Command)

Bits	Field	Access	Reset	Description
[31]	<code>req</code>	RWI	0	Write command update request. Write <code>1</code> to submit. Hardware clears after ACK.
[30]	<code>ack</code>	RO	0	Write command acknowledge.
[29:16]	—	—	0	Reserved.
[15:0]	<code>cmd</code>	RW	0	Write command opcode. Width governed by <code>ctrl.cmd_2bytes</code> .

Typical values:

PSRAM mode	Opcode	cmd_2bytes
SPI write	0x02	0
QSPI write	0x38	0

0x10 — `wr_dummy` (Write Dummy Count)

Bits	Field	Access	Reset	Description
[31:0]	<code>num</code>	RW	0	Dummy cycle count for write transactions. The full 32-bit value is stored; the FSM currently consumes only bits [3:0].

FSM scaling:

Mode	Effective dummy cycles
SPI	<code>num[3:0]</code>

Mode	Effective dummy cycles
QSPI	<code>num[3:0] >> 2</code> (integer divide by 4)

The 32-bit register width is reserved for future extended dummy cycle configurations. For current implementations, only program bits [3:0] unless otherwise documented.

0x14 — `rd_dummy` (Read Dummy Count)

Bits	Field	Access	Reset	Description
[31:0]	<code>num</code>	RW	0	Dummy cycle count for read transactions. Same scaling rules as <code>wr_dummy</code> .

FSM scaling: Identical to `wr_dummy` :

Mode	Effective dummy cycles
SPI	<code>num[3:0]</code>
QSPI	<code>num[3:0] >> 2</code>

0x18 — `mode_status` (Mode Status)

Bits	Field	Access	Reset	Description
[31:2]	—	—	0	Reserved.
[1:0]	<code>current</code>	RO	0	Current PSRAM interface mode, reflected combinationally from the FSM. <code>2'b00</code> =SPI, <code>2'b01</code> =reserved, <code>2'b10</code> =QSPI, <code>2'b11</code> =reserved.

This register is read-only and reflects the live mode output from the QSPI FSM (`o_csr_mode_status_current`). It updates immediately when the FSM transitions mode (e.g., after an independent command completes).

0x1C — `independent` (Independent Command)

Bits	Field	Access	Reset	Description
[31]	<code>req</code>	RWI	0	Independent command request. Write <code>1</code> to issue. Hardware clears after ACK.
[30]	<code>ack</code>	RO	0	Independent command acknowledge. Asserts when FSM has accepted and started the command. Clears automatically.
[29:28]	<code>mode</code>	RW	0	Target interface mode after this command completes. <code>2'b00</code> = SPI, <code>2'b10</code> = QSPI.
[27:16]	—	—	0	Reserved.
[15:0]	<code>cmd</code>	RW	0	Command opcode to send (no address or data phase). Width governed by <code>ctrl.cmd_2bytes</code> .

Notes: - An independent command has only a command phase (and optionally a dummy phase based on the current dummy count). No address or data is sent. - If the FSM is processing AXI transactions when `req` is asserted, it will complete all in-flight transactions before servicing the independent command. `ack` will not assert until the FSM is free. - After `ack` asserts, `mode_status.current` will update to reflect the new mode.

Register Access Sequence (req/ack Pattern)

All registers that use the `req/ack` pattern follow this software sequence:

1. Write the desired field values (`cmd`, `addr`, `data`, `mode`, etc.)
2. Set `req` = 1 in the same or a subsequent APB write
3. Poll the register: read until `ack` = 1
4. Both `req` and `ack` clear automatically — no explicit clear needed
5. Proceed with the next operation

Do not issue AXI transactions that depend on the new configuration until `ack` has been observed.

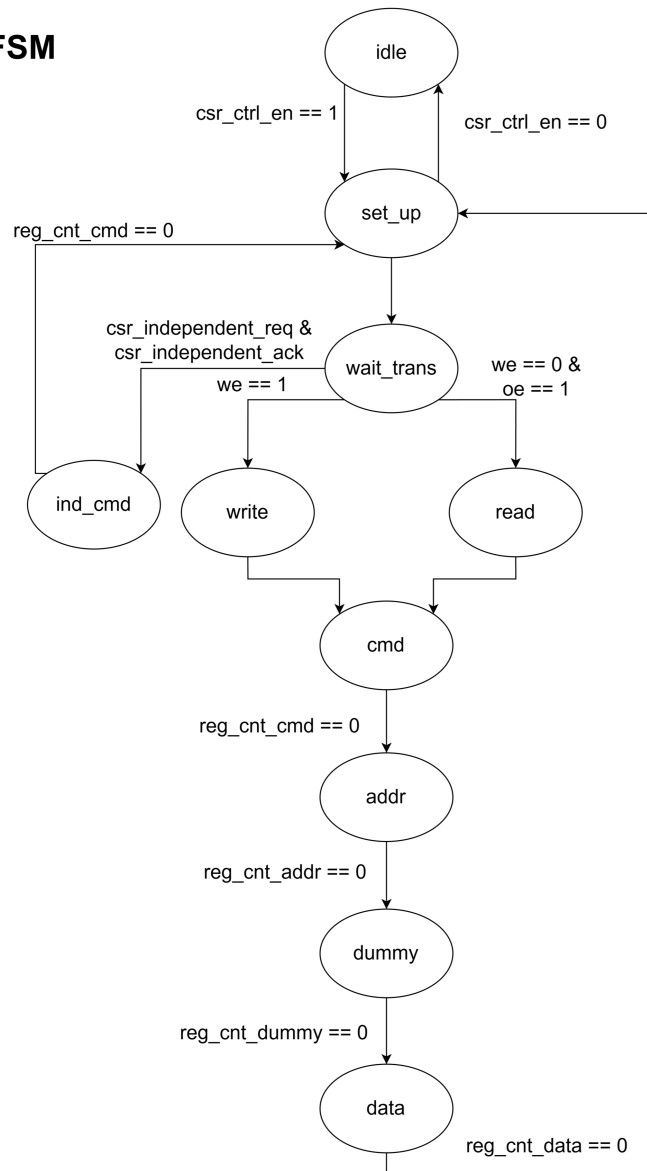
06 — Functional Description

6.1 QSPI FSM

The QSPI FSM (`m_vlsi_qspi_fsm`) is the core transaction engine, running entirely in the SCLK domain. It sequences all PSRAM commands on the physical QSPI interface.

Idle SW can configure all CSR register, and configure to enable the <code>csr_ctrl_en</code> at the last.
Set_up Check the all "req" bits, if the <code>csr_*_req</code> and there is no remain items (FIFO read and write are empty), IP will set the <code>csr_*_ack</code> = 1
Wait_trans If <code>csr_independent_req</code> and <code>ack</code> == 1, IP will move to "Ind_cmd" state, unless IP wait transaction for read and write
Ind_cmd CS is change 1 - 0, <code>reg_cnt_cmd</code> counts down, command is sent to IOPAD, when the <code>reg_cnt_cmd</code> == 0, IP returns to "set_up" state
Write/Read set the corresponding register <code>reg_in_write/reg_in_read</code> , which are used to mode data from FIFO to IOPAD in write phase or shift data from IOPAD to internal register in read phase
Cmd CS changes 1-0 and <code>reg_cnt_cmd</code> is set to <code>csr</code> value before this state. Cmd is sent out to IOPAD until the <code>reg_cnt_cmd</code> = 0
Addr <code>reg_cnt_addr</code> is set to <code>csr</code> value before this state. Addr is sent out to IOPAD until the <code>reg_cnt_addr</code> = 0
Dummy <code>reg_cnt_dummy</code> is set to <code>csr</code> value before this state. the value of IOPAD will be dummy until the <code>reg_cnt_dummy</code> = 0
Data <code>reg_cnt_data</code> is set to <code>csr</code> value before this state. the data will shift out to IOPAD until the <code>reg_cnt_data</code> = 0

FSM



6.1.1 State Summary

State	Description
<code>S_IDLE</code>	Waiting for a request from the arbiter. CS is deasserted.
<code>S_SET_UP</code>	Captures CSR snapshot (command opcode, data/address width, dummy count). Acknowledges pending CSR <code>req</code> bits.
<code>S_WAIT_TRANS</code>	Asserts <code>o_ax_write_valid</code> to accept write data from SCLK-side AXI logic. Waits until write data is available.
<code>S_IND_CMD</code>	Sends an independent (command-only) transaction. No address or data phase.
<code>S_CMD</code>	Drives the command phase on QSPI pads. Shifts out 8 or 16 bits depending on <code>ctrl.cmd_2bytes</code> .
<code>S_ADDR</code>	Drives the address phase on QSPI pads. Number of bits set by <code>wd.addr</code> .
<code>S_DUMMY</code>	Drives the dummy phase. Data pins tri-stated. Cycle count derived from <code>wr_dummy / rd_dummy</code> scaled by mode.
<code>S_WRITE</code>	Drives write data onto QSPI pads. Number of bits set by <code>wd.data</code> (or <code>PARAM_DATA_WD-1</code> if <code>auto_data_wd</code>).
<code>S_READ</code>	Samples read data from QSPI pads into shift register. Returns data to SCLK-side AXI logic.

6.1.2 State Transitions

```

S_IDLE
├─ [independent req & no pending AXI] → S_SET_UP → S_IND_CMD → S_IDLE
├─ [write granted by arbiter] → S_SET_UP → S_WAIT_TRANS → S_CMD → S_ADDR → S_DUMMY → S_WRITE → S_READ → S_IDLE
└─ [read granted by arbiter] → S_SET_UP → S_CMD → S_ADDR → S_DUMMY → S_READ → S_IDLE

```

- CS (`o_qspi_csn`) asserts at the start of `S_CMD` or `S_IND_CMD` and deasserts at the end of the final data/command phase.
- If `ctrl.en = 0`, the FSM stays in `S_IDLE` and ignores all requests.

6.1.3 Phase Details

Command Phase (`S_CMD`, `S_IND_CMD`): - SPI mode: DQ[0] driven, DQ[1:3] tri-stated - QSPI mode: DQ[3:0] all driven - 8 bits sent if `ctrl.cmd_2bytes = 0`; 16 bits (2 bytes, MSB first) if `ctrl.cmd_2bytes = 1`

Address Phase (`s_addr`): - SPI mode: DQ[0] driven serially - QSPI mode: DQ[3:0] driven (4 bits per clock) - Number of bits: `wd.addr`

Dummy Phase (`s_dummy`): - All DQ pins tri-stated - Cycle count: `rd_dummy.num[3:0]` (read) or `wr_dummy.num[3:0]` (write), scaled by mode: - SPI: value used directly - QSPI: value divided by 4

Data Phase (`s_write` / `s_read`): - SPI write: DQ[0] driven serially - QSPI write: DQ[3:0] driven (4 bits per clock) - SPI read: DQ[1] sampled serially - QSPI read: DQ[3:0] sampled (4 bits per clock) - Number of bits: `wd.data` (or `PARAM_DATA_WD-1` when `auto_data_wd = 1`)

6.2 Arbitration

The arbiter (`m_vlsi_arbiter`) performs round-robin selection between concurrent read and write requests in the SCLK domain.

6.2.1 Arbitration Logic

`arb_toggle` flips after each granted transaction, ensuring neither read nor write can starve.

6.2.2 Request Conditions

- A write request requires both an address (AWFIFO) and data (WFIFO) beat to be available, and space in BFIFO for the response.
 - A read request requires an address (ARFIFO) beat, no read already in flight (`rd_pending`), and space in RFIFO for the response.
-

6.3 Write Flow

1. **AXI address channel:** `m_vlsi_axfsm` (AW instance) accepts the AXI burst descriptor, generates per-beat addresses for the burst, and pushes each `{aw_last, aw_id, aw_addr}` entry into AWFIFO.
2. **AXI data channel:** `m_vlsi_axi_bus_logic` accepts W-channel beats and pushes `{w_last, w_data}` into WFIFO.
3. **Arbitration:** The arbiter grants a write when `req_write` is asserted. `m_vlsi_axi_sclk_logic` pops AWFIFO and WFIFO and presents the data to the FSM.
4. **FSM sequencing:** `S_SET_UP` → `S_WAIT_TRANS` → `S_CMD` → `S_ADDR` → `S_DUMMY` → `S_WRITE`

5. **Response:** After the last write beat, `{aw_id, bresp}` is pushed into BFIFO. `m_vlsi_axi_bus_logic` pops BFIFO and drives the AXI B channel.

Write response codes:

<code>o_bresp</code>	Condition
<code>2'b00</code> (OKAY)	AW burst length matches W beat count
<code>2'b10</code> (SLVERR)	<code>aw_last</code> and <code>w_last</code> mismatch — malformed burst

6.4 Read Flow

1. **AXI address channel:** `m_vlsi_axfsm` (AR instance) generates per-beat addresses and pushes `{ar_last, ar_id, ar_addr}` into ARFIFO.
2. **Arbitration:** The arbiter grants a read when `req_read` is asserted. `m_vlsi_axi_sclk_logic` pops ARFIFO and presents the address to the FSM.
3. **FSM sequencing:** `S_SET_UP → S_CMD → S_ADDR → S_DUMMY → S_READ`
4. **Data return:** Read data is pushed into RFIFO as `{rd_id, 2'b00, rdata, rd_last}`.
5. **AXI data channel:** `m_vlsi_axi_bus_logic` pops RFIFO and drives the AXI R channel.

6.5 Independent Command Flow

An independent command is a command-only transaction — no address or data phase. It is primarily used to switch the PSRAM from SPI to QSPI mode (or vice versa).

Sequence: 1. Software writes `independent.cmd` and `independent.mode`, then sets `independent.req = 1`. 2. If the FSM has in-flight AXI transactions, it completes them first. 3. FSM enters `S_IND_CMD`, drives the command on QSPI pads, then updates the internal interface mode register to `independent.mode`. 4. FSM asserts `independent.ack`. Both `req` and `ack` clear automatically. 5. `mode_status.current` updates to reflect the new mode.

After a SPI→QSPI mode switch, the user should update `read.cmd` and `write.cmd` to the QSPI equivalents before issuing further AXI transactions.

6.6 XIP Mode

XIP (Execute-In-Place) behavior is inferred from the AXI burst length — no dedicated CSR bit is required.

AXI transaction type	FSM behavior
Single-beat (awlen/arlen = 0)	Standard single-word read or write
Multi-beat burst (awlen/arlen > 0)	Streaming (XIP) read or write — address auto-increments per beat

The user must ensure the PSRAM device is in the correct operating mode (e.g., QSPI) before issuing burst transactions intended for XIP.

6.7 auto_data_wd Mode

When `ctrl.auto_data_wd = 1`, the FSM ignores the `wd.data` register and instead forces the data width to `PARAM_DATA_WD - 1` on every `S_WRITE` or `S_READ` entry. This is useful when the PSRAM always transfers full-width words and per-transaction width control is not needed.

6.8 Write Response Error (SLVERR)

`BRESP = SLVERR (2'b10)` is generated when:

```
(aw_last from AWFIFO) XOR (w_last from WFIFO) ≠ 0
```

This condition detects a mismatch between the declared AW burst length and the actual number of W-channel beats received. It protects the system against malformed bursts that could leave the FSM in an inconsistent state.

07 — Software Integration Guide

7.1 Initialization Sequence

Perform this sequence after all clocks are stable and before issuing any AXI transactions.

```
Step 1: Assert all resets (hold i_qspi_rstn, i_axi_rstn, i_apb_rstn low)
Step 2: Start all clocks and wait for stability (PLL lock, etc.)
Step 3: De-assert i_qspi_rstn first; wait  $\geq 2$  SCLK cycles
Step 4: De-assert i_axi_rstn and i_apb_rstn
Step 5: Configure registers via APB (see below)
Step 6: Set ctrl.en = 1
```

7.1.1 Register Configuration Order

```
a. Write wr_dummy.num - dummy cycles for writes
b. Write rd_dummy.num - dummy cycles for reads

c. Write write.cmd, then set write.req = 1
   Poll write.ack until asserted

d. Write read.cmd, then set read.req = 1
   Poll read.ack until asserted

e. Write wd.addr and wd.data, then set wd.req = 1
   Poll wd.ack until asserted

f. If the PSRAM starts in SPI mode and QSPI is desired:
   Write independent.cmd = <PSRAM QPI-enable opcode>
   Write independent.mode = 2'b10 (QSPI)
   Set independent.req = 1
   Poll independent.ack until asserted

g. Write ctrl.en = 1
```

`wr_dummy` and `rd_dummy` do not use the req/ack mechanism — they are direct register writes that take effect on the next transaction.

7.2 Normal AXI Read/Write

After initialization, the CPU issues standard AXI4 transactions. No additional software intervention is required per transaction.

- AXI address maps **directly** to the PSRAM physical address — no offset or shift applied.
 - Burst types FIXED, INCR, and WRAP are all supported.
 - Monitor `o_bresp` on the B channel; `SILVERR (2'b10)` indicates a burst-length mismatch (see [06 functional.md §6.8](#)).
-

7.3 Changing Command Opcodes at Run Time

To switch to a different command opcode (e.g., temporarily using Read-ID):

1. Write the new opcode to `read.cmd` (or `write.cmd`)
2. Set `read.req = 1` (or `write.req = 1`)
3. Poll `read.ack` until asserted
4. Issue the special AXI transaction
5. Write the standard opcode back; repeat steps 1-3 to restore

If the address or data width also needs to change for this command:

1. Write `wd.addr` and/or `wd.data` to the new values
2. Set `wd.req = 1`
3. Poll `wd.ack` until asserted
4. Issue the transaction
5. Restore `wd.addr/wd.data` and repeat the `req/ack` cycle

Do not issue AXI transactions between setting `req` and observing `ack`. The new configuration is not guaranteed to be active in the SCLK domain until `ack` asserts.

7.4 Independent (Command-Only) Transaction

Used for PSRAM control commands that have no data payload — e.g., mode switching, reset, burst-length configuration.

1. Write `independent.cmd = <opcode>`
2. Write `independent.mode = target mode after this command`
(`2'b00 = SPI`, `2'b10 = QSPI`)
3. Set `independent.req = 1`
4. Poll `independent.ack` until asserted
(Any pending AXI transactions complete first; `ack` may be delayed)

5. req and ack clear automatically – no explicit write needed
6. Read mode_status.current to confirm the new mode

7.5 Switching from SPI to QSPI Mode

Full sequence for transitioning a PSRAM device from SPI to QSPI mode:

1. Confirm the PSRAM is currently in SPI mode
(default after power-on or reset for most devices)
2. Issue the mode-switch independent command:
independent.cmd = <device QPI-enter opcode, e.g. 0x35>
independent.mode = 2'b10
independent.req = 1
→ poll independent.ack
3. Update read and write command opcodes:
read.cmd = <QSPI read opcode, e.g. 0xEB>
read.req = 1 → poll read.ack
write.cmd = <QSPI write opcode, e.g. 0x38>
write.req = 1 → poll write.ack
4. Update dummy cycle counts if needed for the new mode:
rd_dummy.num = <new value>
wr_dummy.num = <new value>
5. Resume normal AXI transactions (now in QSPI mode)

7.6 Switching from QSPI to SPI Mode

1. Issue the mode-switch independent command:
independent.cmd = <device SPI-exit opcode, e.g. 0xF5>
independent.mode = 2'b00
independent.req = 1
→ poll independent.ack
2. Update read and write command opcodes to SPI values:
read.cmd = <SPI read opcode, e.g. 0x03>
read.req = 1 → poll read.ack
write.cmd = <SPI write opcode, e.g. 0x02>
write.req = 1 → poll write.ack
3. Update dummy cycle counts if needed
4. Resume normal AXI transactions (now in SPI mode)

7.7 APB Access Latency Notes

Every APB register access (read or write) stalls on `PREADY` for the duration of the 4-phase CDC handshake.

```
Minimum stall  $\approx 2 \times T_{\text{SCLK}} + 2 \times T_{\text{PCLK}}$ 
```

Example: SCLK = 100 MHz (T = 10 ns), PCLK = 50 MHz (T = 20 ns):

```
Minimum stall  $\approx 2 \times 10 + 2 \times 20 = 60 \text{ ns} \approx 3 \text{ PCLK cycles}$ 
```

Software must not assume zero-latency APB access. The APB infrastructure handles the stall transparently if `PREADY` is wired to the bus controller. In bare-metal drivers, ensure the APB read function waits for `PREADY` before capturing `PRDATA`.

7.8 Polling vs. Interrupt-Driven Operation

The IP does not generate interrupts. All req/ack synchronization is polling-based:

- For initialization sequences, simple polling loops are sufficient since the latency is short (tens of nanoseconds per access).
 - For high-frequency runtime command changes, consider using a DMA or hardware sequencer to avoid polling overhead in latency-sensitive paths.
-

7.9 Bare-Metal Driver Pseudocode

```
// Write a CSR register with req/ack
void csr_write_with_ack(uint32_t offset, uint32_t value) {
    apb_write(BASE + offset, value | (1u << 31)); // set req
    while (!(apb_read(BASE + offset) & (1u << 30))) // poll ack
        ;
}

// Initialize the IP for QSPI operation
void qspi_psram_init(void) {
    apb_write(BASE + 0x10, 0); // wr_dummy = 0
    apb_write(BASE + 0x14, 6); // rd_dummy = 6 (QSPI: 6>>2 = 1 cycle)

    csr_write_with_ack(0x0C, 0x38); // write.cmd = 0x38 (QSPI write)
    csr_write_with_ack(0x08, 0xEB); // read.cmd = 0xEB (QSPI read)
    csr_write_with_ack(0x04,
        (24u << 24) | 32u); // wd: addr=24 bits, data=32 bits
}
```



```
// Issue SPI→QSPI mode switch command (e.g., 0x35 for some PSRAMs)
csr_write_with_ack(0x1C,
    (0x35u) | (0x2u << 28));          // cmd=0x35, mode=QSPI

apb_write(BASE + 0x00, 0x1);          // ctrl.en = 1
}
```

08 — STA Constraints Reference

This section lists all timing constraints required for correct static timing analysis of the IP. Apply these constraints in your synthesis and STA scripts.

8.1 ACLK ↔ SCLK (Async FIFO Toggle-Vector Synchronizers)

These constraints apply to the `m_vlsi_multi_synch` instances inside each `m_vlsi_async_fifo`. They cover the occupancy vector paths that cross between ACLK and SCLK.

```
# ACLK → SCLK: write-side vector to read side
set_max_delay \
    -from [get_clocks {i_axi_clk}] \
    -to   [get_clocks {i_qspi_clk}] \
    [expr {1.5 * $SCLK_period}] \
    -datapath_only

# SCLK → ACLK: read-side vector to write side
set_max_delay \
    -from [get_clocks {i_qspi_clk}] \
    -to   [get_clocks {i_axi_clk}] \
    [expr {1.5 * $ACLK_period}] \
    -datapath_only
```

The delay value is **1.0 to 1.5 times the receiving clock period**. Using 1.5× provides margin for synchronizer setup time.

8.2 PCLK ↔ SCLK (CSR 4-Phase Handshake Synchronizers)

These constraints apply to the `m_vlsi_synch` instances inside `m_vlsi_qspi_csr`. They cover the req/ack handshake signals crossing between PCLK and SCLK.

```
# PCLK → SCLK: request signals from APB domain to register domain
set_max_delay \
    -from [get_clocks {i_apb_clk}] \
    -to   [get_clocks {i_qspi_clk}] \
    [expr {1.5 * $SCLK_period}] \
```

```
# SCLK → PCLK: acknowledge signals from register domain back to APB domain
set_max_delay \
    -from [get_clocks {i_qspi_clk}] \
    -to [get_clocks {i_apb_clk}] \
    [expr {1.5 * $PCLK_period}] \
```

8.3 False Path Declarations

ACLK and PCLK never directly exchange data — all AXI↔APB paths are either through independent logic or through the SCLK domain as the intermediate. These false paths prevent STA tools from trying to analyze a nonexistent relationship.

8.4 Reset Path Constraints

All resets are synchronous within their domain. No special `set_false_path` for reset paths is required unless the tool flags asynchronous reset removal paths across domains.

If the tool reports a CDC violation on reset paths:

```
# Reset paths are synchronous within each domain — no cross-domain reset paths exist
set_false_path -from [get_ports {i_axi_rstn}] -to [get_clocks {i_qspi_clk}]
set_false_path -from [get_ports {i_axi_rstn}] -to [get_clocks {i_apb_clk}]
set_false_path -from [get_ports {i_apb_rstn}] -to [get_clocks {i_axi_clk}]
set_false_path -from [get_ports {i_apb_rstn}] -to [get_clocks {i_qspi_clk}]
set_false_path -from [get_ports {i_qspi_rstn}] -to [get_clocks {i_axi_clk}]
set_false_path -from [get_ports {i_qspi_rstn}] -to [get_clocks {i_apb_clk}]
```

8.5 Synchronizer Cell Attributes

To ensure synthesis does not optimize away synchronizer flip-flop stages, apply the `DONT_TOUCH` or equivalent attribute to all `m_vlsi_synch` and `m_vlsi_multi_synch`, please replace synchronizer cell with your library, unless you must make sure they will not be optimized during the synthesis phase.

```
# Synopsys DC / Fusion Compiler
set_dont_touch [get_cells -hierarchical -filter {REF_NAME == m_vlsi_synch}]
set_dont_touch [get_cells -hierarchical -filter {REF_NAME == m_vlsi_multi_synch}]
```

Or in RTL (if synthesis attributes are used):

```
(* dont_touch = "true" *) m_vlsi_synch u_synch ( ... );
```

8.6 Constraint Summary Table

Path	Constraint type	Value
ACLK → SCLK (FIFO write)	set_max_delay	$1.5 \times T_{SCLK}$
SCLK → ACLK (FIFO read)	set_max_delay	$1.5 \times T_{ACLK}$
PCLK → SCLK (CSR write)	set_max_delay	$1.5 \times T_{SCLK}$
SCLK → PCLK (CSR read)	set_max_delay	$1.5 \times T_{PCLK}$

8.7 CDC Tool Waiver Guidance

After applying the above constraints, all remaining CDC violations can be waived. The release provides CDC folder with CDC waiver file, which can be used as reference

QSPI/SPI PSRAM Controller — Hardware Design Specification

Revision: 2.0 **Date:** 2026-06 **Author:** Ethan, Link **Page:** VLSI Technology

I. Overview

The QSPI PSRAM Controller is a hardware IP developed by **VLSI Technology** that bridges an AXI4 memory-mapped bus to an external PSRAM device over a QSPI (Quad SPI) or standard SPI interface. The IP accepts AXI4 read and write transactions from a host, translates them into the command-address-dummy-data phase sequence required by QSPI/SPI PSRAMs, and drives the I/O pads accordingly. A separate APB slave interface provides run-time configuration of command values, data widths, dummy-cycle counts, and operating mode through a set of Control and Status Registers (CSRs).

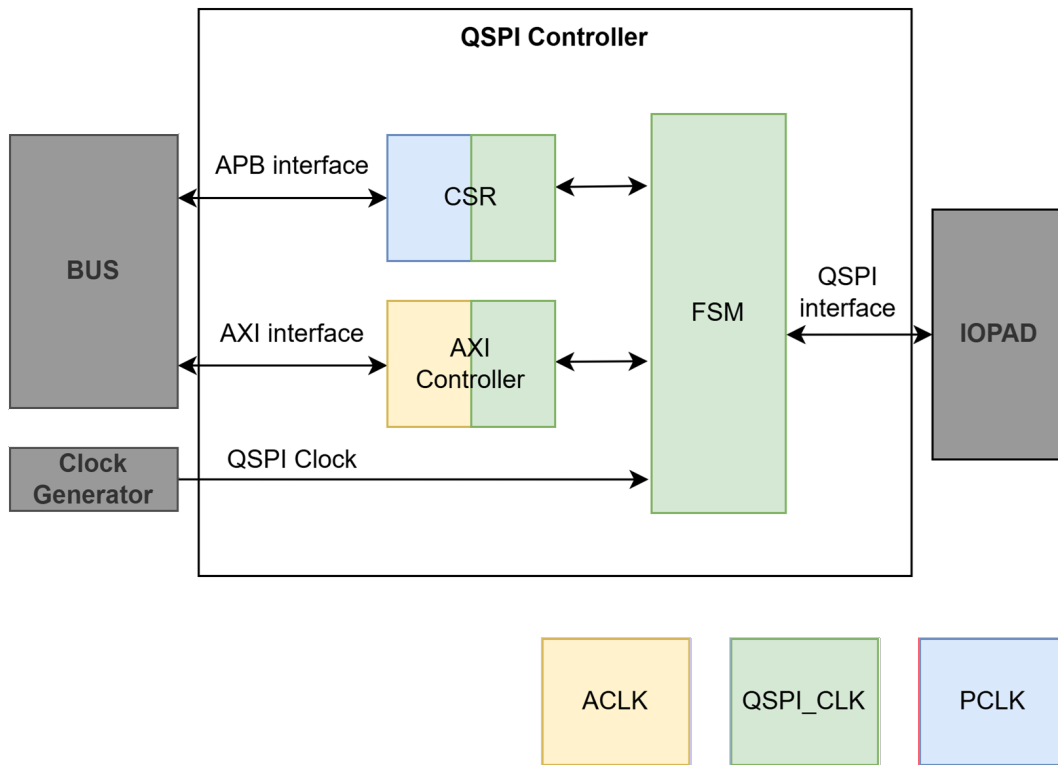
Key capabilities:

- AXI4 full slave interface (AW, W, B, AR, R channels) with burst support (FIXED, INCR, WRAP)
- APB slave interface for CSR configuration
- QSPI (4-bit I/O) and SPI (1-bit I/O) protocol support
- Software-controlled independent commands (e.g., mode switching)
- Dynamic reconfiguration of read/write commands, address/data widths, and dummy cycles
- XIP (Execute-In-Place) support inferred from AXI burst semantics
- Round-robin arbitration between concurrent read and write requests

Top-level parameters:

Parameter	Default	Description
<code>PARAM_AXI_DATA_WD</code>	32	AXI data bus width (bits)
<code>PARAM_AXI_ADDR_WD</code>	24	AXI address bus width (bits)
<code>PARAM_AXI_ID_WD</code>	4	AXI ID field width (bits)
<code>PARAM_AXI_LEN_WD</code>	8	AXI burst length field width (bits)
<code>PARAM_AXI_FIFO_DEPTH</code>	8	Depth of each async FIFO

II. Block Diagram



The controller comprises five major sub-systems:

Sub-system	Module(s)	Clock domain	Role
AXI Controller	<code>m_vlsi_axfsm</code> ×2, <code>m_vlsi_axi_bus_logic</code> , <code>m_vlsi_axi_sclk_logic</code> , <code>m_vlsi_arbiter</code> , <code>m_vlsi_async_fifo</code> ×5	ACLK / SCLK	Accepts AXI transactions, crosses to SCLK, drives PSRAM interface
CSR	<code>m_vlsi_qspi_csr</code>	PCLK / SCLK	APB register file with 4-phase handshake CDC
QSPI FSM	<code>m_vlsi_qspi_fsm</code>	SCLK	Transaction engine: CMD-ADDR-DUMMY-DATA sequencing
Synchroniser	<code>m_vlsi_synch</code> , <code>m_vlsi_multi_synch</code>	Various	2-stage flip-flop synchronisers used

Sub-system	Module(s)	Clock domain	Role
			by FIFO CDC and CSR CDC
Arbiter	m_vlsi_arbiter	SCLK	Round-robin read/write arbitration

III. Port Description

3.1 Clock and Reset

Port	Direction	Width	Description
i_axi_clk	Input	1	AXI bus clock (ACLK)
i_axi_rstn	Input	1	AXI active-low synchronous reset
i_apb_clk	Input	1	APB bus clock (PCLK)
i_apb_rstn	Input	1	APB active-low synchronous reset
i_qspi_clk	Input	1	QSPI/internal logic clock (SCLK). Also directly drives o_qspi_sclk.
i_qspi_rstn	Input	1	QSPI active-low synchronous reset

3.2 AXI4 Slave Interface

Write Address Channel (AW)

Port	Direction	Width	Description
i_awvalid	Input	1	Write address valid
o_awready	Output	1	Write address ready
i_awaddr	Input	PARAM_AXI_ADDR_WD	Write address
i_awid	Input	PARAM_AXI_ID_WD	Write transaction ID
i_awlen	Input	PARAM_AXI_LEN_WD	Burst length minus 1
i_awsz	Input	3	Burst size
i_awburst	Input	2	

Port	Direction	Width	Description
			Burst type: 00 =FIXED, 01 =INCR, 10 =WRAP

Write Data Channel (W)

Port	Direction	Width	Description
i_wvalid	Input	1	Write data valid
o_wready	Output	1	Write data ready
i_wdata	Input	PARAM_AXI_DATA_WD	Write data
i_wstrb	Input	PARAM_AXI_DATA_WD/8	Write strobes
i_wlast	Input	1	Last beat of burst

Write Response Channel (B)

Port	Direction	Width	Description
o_bvalid	Output	1	Write response valid
i_bready	Input	1	Write response ready
o_bid	Output	PARAM_AXI_ID_WD	Write response ID
o_bresp	Output	2	Write response: 00 =OKAY, 10 =SLVERR

Read Address Channel (AR)

Port	Direction	Width	Description
i_arvalid	Input	1	Read address valid
o_arready	Output	1	Read address ready
i_araddr	Input	PARAM_AXI_ADDR_WD	Read address
i_arid	Input	PARAM_AXI_ID_WD	Read transaction ID
i_arlen	Input	PARAM_AXI_LEN_WD	Burst length minus 1
i_arsize	Input	3	Burst size

Port	Direction	Width	Description
i_arburst	Input	2	Burst type: 00 =FIXED, 01 =INCR, 10 =WRAP

Read Data Channel (R)

Port	Direction	Width	Description
o_rvalid	Output	1	Read data valid
i_rready	Input	1	Read data ready
o_rdata	Output	PARA_AXI_DATA_WD	Read data
o_rid	Output	PARA_AXI_ID_WD	Read response ID
o_rlast	Output	1	Last beat of burst
o_rresp	Output	2	Read response: 00 =OKAY

3.3 APB Slave Interface

Port	Direction	Width	Description
i_psel	Input	1	APB select
i_penable	Input	1	APB enable
i_pwrite	Input	1	APB write (1) / read (0)
i_paddr	Input	16	APB address
i_pwdata	Input	32	APB write data
i_pstrb	Input	4	APB write strobes
i_pprot	Input	3	APB protection attributes
i_protect_en	Input	1	Enable protection check on pprot[1]
i_slverr_en	Input	1	Enable SLVERR on address/strobe violations
o_prdata	Output	32	APB read data
o_pready	Output	1	APB ready (stalls bus while CDC in progress)

Port	Direction	Width	Description
<code>o_pslverr</code>	Output	1	APB slave error

3.4 QSPI Pad Interface

Port	Direction	Width	Description
<code>o_qspi_sclk</code>	Output	1	QSPI clock output — directly wired from <code>i_qspi_clk</code>
<code>o_qspi_csn</code>	Output	1	QSPI chip-select (active low)
<code>io_qspi_data</code>	Inout	4	Quad SPI data lines: DQ[3:0]

Note: `o_qspi_sclk` is a direct wire assignment (`assign o_qspi_sclk = i_qspi_clk`). There is no clock gating on the SCLK output.

IV. Clocks and Resets

The IP operates across **three independent, asynchronous clock domains**:

Clock	Reset	Domain name	Used by
<code>i_axi_clk</code>	<code>i_axi_rstn</code>	ACLK	AXI4 bus logic, async FIFO write side (AW, W, AR) and read side (B, R)
<code>i_apb_clk</code>	<code>i_apb_rstn</code>	PCLK	APB CSR interface, 4-phase handshake bus side
<code>i_qspi_clk</code>	<code>i_qspi_rstn</code>	SCLK	QSPI FSM, register file, async FIFO SCLK side, 4-phase handshake reg side

All resets are active-low. Each domain has its own independent reset and must be held in reset until its clock is stable.

`o_qspi_sclk` is tied directly to `i_qspi_clk` with no gating; the clock runs continuously whenever the IP is powered.

V. Clock Domain Crossing (CDC)

5.1 ACLK ↔ SCLK: Asynchronous FIFO (Toggle-Vector Scheme)

The AXI Controller uses five asynchronous FIFOs (`m_vlsi_async_fifo`) to cross between ACLK and SCLK:

FIFO	Direction	Contents
AWFIFO	ACLK → SCLK	Write address: {aw_last, aw_id, aw_addr}
WFIFO	ACLK → SCLK	Write data: {w_last, w_data}
ARFIFO	ACLK → SCLK	Read address: {ar_last, ar_id, ar_addr}
BFIFO	SCLK → ACLK	Write response: {aw_id, bresp}
RFIFO	SCLK → ACLK	Read response: {rd_id, 2'b00, rdata, rd_last}

CDC scheme — Toggle-Vector (not Gray-encoded pointers):

Each FIFO uses a `PARAM_DEPTH`-wide shift register (`reg_wval` on the write side, `reg_rval` on the read side) to track occupancy. On each write, the write-side vector shifts left and inverts the MSB. The same operation occurs on each read on the read side. Occupancy is determined by XOR/XNOR comparison between the local vector and the synchronized copy of the opposite side's vector.

Write side:

```
reg_wval <= {reg_wval[DEPTH-2:0], ~reg_wval[DEPTH-1]}; // shift + invert MSB on every wr
wr_comb_val = reg_wval ~^ rval_sync; // XNOR with synced read vector
o_write_valid = wr_comb_val[reg_wr_cnt]; // slot available when bits mat
```

Read side:

```
reg_rval <= {reg_rval[DEPTH-2:0], ~reg_rval[DEPTH-1]}; // shift + invert MSB on every r
rd_comb_val = reg_rval ^ wval_sync; // XOR with synced write vector
o_read_valid = rd_comb_val[reg_rd_cnt]; // data available when bits dif
```

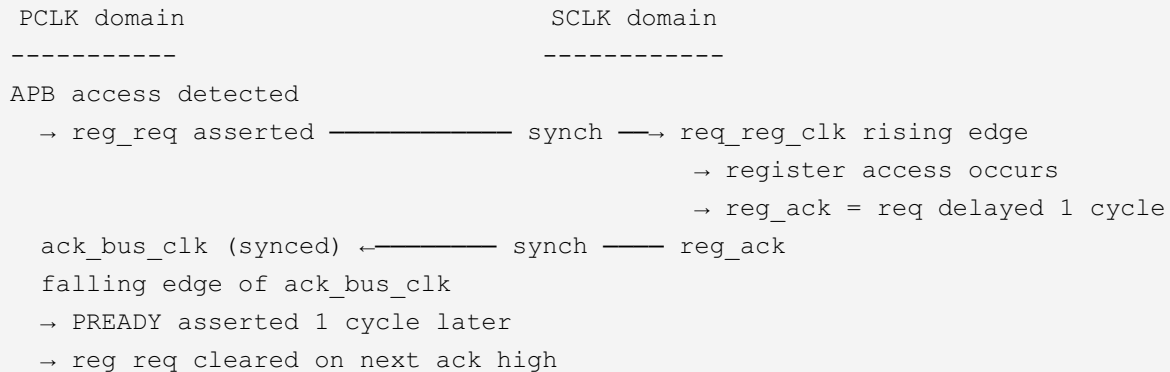
Each direction uses a `m_vlsi_multi_synch` instance (`PARAM_DEPTH`-wide 2-stage synchronizer) to safely transfer the vector to the opposite clock domain.

Timing constraints required:

```
set_max_delay -from [get_clocks ACLK] -to [get_clocks SCLK] <1.5 × SCLK_period>
set_max_delay -from [get_clocks SCLK] -to [get_clocks ACLK] <1.5 × ACLK_period>
```

5.2 PCLK ↔ SCLK: 4-Phase Handshake (CSR)

The CSR module crosses APB register accesses into the SCLK domain using a 4-phase handshake. The mechanism is identical for reads and writes:



Key signals in `m_vlsi_qspi_csr`:

- `reg_read_req` / `reg_write_req`: request flops in PCLK domain
- `read_req_reg_clk` / `write_req_reg_clk`: synchronized request in SCLK domain (rising edge = trigger)
- `read_ack` / `write_ack`: delayed copy of synchronized request, used as ACK back to PCLK domain
- `read_ack_bus_clk` / `write_ack_bus_clk`: ACK synchronized back to PCLK
- `PREADY` is asserted one PCLK cycle after the falling edge of the synchronized ACK

The APB bus is stalled (`PREADY` deasserted) for the full round-trip CDC latency (approximately $2 \times \text{SCLK} + 2 \times \text{PCLK}$ cycles minimum).

Timing constraints required:

```
set_max_delay -from [get_clocks PCLK] -to [get_clocks SCLK] <1.5 × SCLK_period>
set_max_delay -from [get_clocks SCLK] -to [get_clocks PCLK] <1.5 × PCLK_period>
```

VI. Configuration Registers (CSR)

All registers are 32 bits wide and accessed through the APB interface. The register file is clocked by SCLK; APB writes and reads are synchronized via the 4-phase handshake described in Section V.2.

Access type key: RW = read/write, RO = read-only, RWI = read/write with hardware write-back (hardware can clear the `req` bit)

0x00 — `ctrl` (Control)

Bits	Field	Access	Reset	Description
[31:3]	—	—	0	Reserved
[2]	<code>cmd_2bytes</code>	RW	0	<code>0</code> : 1-byte (8-bit) command; <code>1</code> : 2-byte (16-bit) command
[1]	<code>auto_data_wd</code>	RW	0	<code>1</code> : automatically set data width to <code>PARAM_DATA_WD-1</code> on each transaction
[0]	<code>en</code>	RW	0	IP enable. Must be set before issuing AXI transactions.

0x04 — `wd` (Width Configuration)

Bits	Field	Access	Reset	Description
[31]	<code>req</code>	RWI	0	Write <code>1</code> to request a width change. Hardware clears after ACK.
[30]	<code>ack</code>	RO	0	Hardware asserts when the width change has been applied in SCLK domain.
[29:24]	<code>addr</code>	RW	0	Address width (6-bit value, unit: bits)
[23:0]	<code>data</code>	RW	0	Data width (24-bit value, unit: bits)

0x08 — `read` (Read Command)

Bits	Field	Access	Reset	Description
[31]	<code>req</code>	RWI	0	Write <code>1</code> to update the read command. Hardware clears after ACK.
[30]	<code>ack</code>	RO	0	Hardware asserts when the new command is latched in SCLK domain.
[29:16]	—	—	0	Reserved
[15:0]	<code>cmd</code>	RW	0	Read command value (8-bit or 16-bit, selected by <code>ctrl.cmd_2bytes</code>)

0x0C — `write` (Write Command)

Bits	Field	Access	Reset	Description
[31]	<code>req</code>	RWI	0	Write <code>1</code> to update the write command. Hardware clears after ACK.
[30]	<code>ack</code>	RO	0	Hardware asserts when the new command is latched in SCLK domain.
[29:16]	—	—	0	Reserved
[15:0]	<code>cmd</code>	RW	0	Write command value (8-bit or 16-bit, selected by <code>ctrl.cmd_2bytes</code>)

0x10 — `wr_dummy` (Write Dummy Count)

Bits	Field	Access	Reset	Description
[31:0]	<code>num</code>	RW	0	Number of dummy cycles for write transactions. Full 32-bit field is stored; the FSM currently consumes only bits [3:0].

Note: In QSPI mode the FSM uses `num[3:0] >> 2`; in SPI mode it uses `num[3:0]` directly. The 32-bit register width is reserved for future support of extended dummy cycle configurations.

0x14 — `rd_dummy` (Read Dummy Count)

Bits	Field	Access	Reset	Description
[31:0]	<code>num</code>	RW	0	Number of dummy cycles for read transactions. Full 32-bit field is stored; the FSM currently consumes only bits [3:0].

Note: Same scaling as `wr_dummy` — QSPI: `num[3:0] >> 2`; SPI: `num[3:0]`.

0x18 — `mode_status` (Mode Status)

Bits	Field	Access	Reset	Description
[31:2]	—	—	0	Reserved
[1:0]	<code>current</code>	RO	0	Current interface mode: <code>00</code> =SPI, <code>01</code> =reserved, <code>10</code> =QSPI, <code>11</code> =reserved.

Bits	Field	Access	Reset	Description
				Reflects <code>i_csr_independent_mode</code> combinationaly from the FSM.

0x1C — `independent` (Independent Command)

Bits	Field	Access	Reset	Description
[31]	<code>req</code>	RWI	0	Write <code>1</code> to issue an independent command. Hardware clears after ACK.
[30]	<code>ack</code>	RO	0	Hardware asserts when the FSM has accepted the independent command.
[29:28]	<code>mode</code>	RW	0	Target interface mode after this command: <code>00</code> =SPI, <code>10</code> =QSPI
[27:16]	—	—	0	Reserved
[15:0]	<code>cmd</code>	RW	0	Command value to send (independent of read/write command registers)

VII. Functional Description

7.1 QSPI FSM States

The QSPI FSM (`m_vlsi_qspi_fsm`) implements the core transaction sequencing in the SCLK domain.

Idle
SW can configure all CSR register, and configure to enable the csr_ctrl_en at the last.

Set_up
Check the all "req" bits, if the csr_*_req and there is no remain items (FIFO read and write are empty), IP will set the csr*_ack = 1

Wait_trans
If csr_independent_req and ack == 1, IP will move to "Ind_cmd" state, unless IP wait transaction for read and write

Ind_cmd
CS is change 1 - 0, reg_cnt_cmd counts down, command is sent to IOPAD, when the reg_cnt_cmd == 0, IP returns to "set_up" state

Write/Read
set the corresponding register
reg_in_write/reg_in_read, which are used to mode data from FIFO to IOPAD in write phase or shift data from IOPAD to internal register in read phase

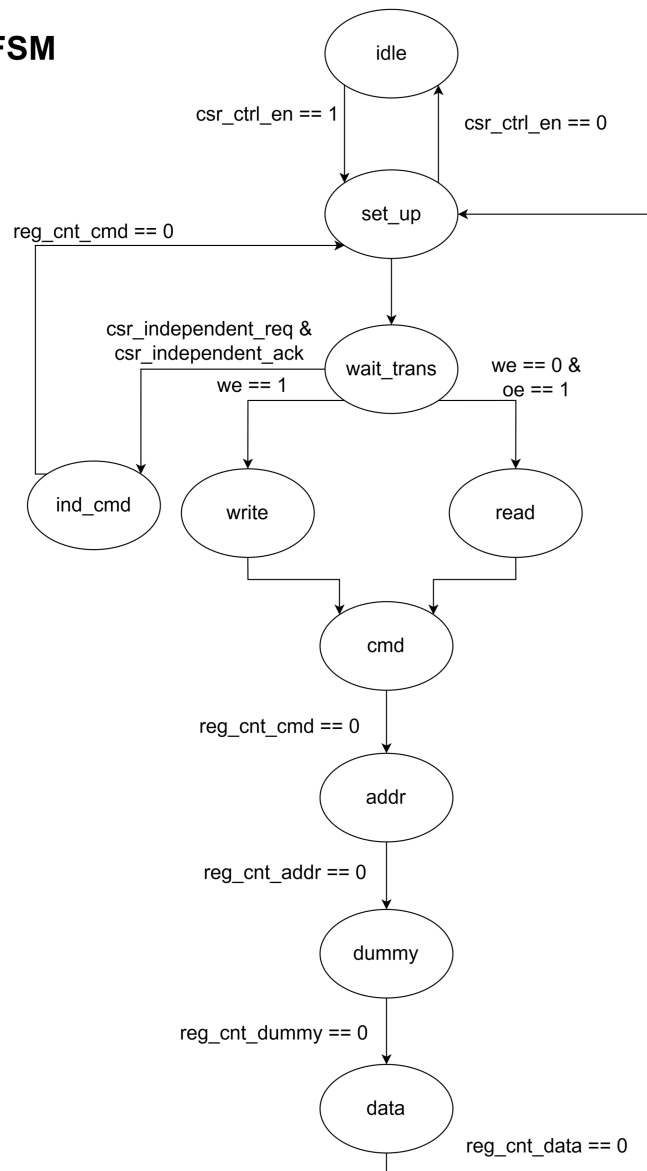
Cmd
CS changes 1-0 and reg_cnt_cmd is set to csr value before this state. Cmd is sent out to IOPAD until the reg_cnt_cmd = 0

Addr
reg_cnt_addr is set to csr value before this state. Addr is sent out to IOPAD until the reg_cnt_addr = 0

Dummy
reg_cnt_dummy is set to csr value before this state. the value of IOPAD will be dummy until the reg_cnt_dummy = 0

Data
reg_cnt_data is set to csr value before this state. the data will shift out to IOPAD until the reg_cnt_data = 0

FSM



State	Description
S_IDLE	Waiting for a request from the arbiter
S_SET_UP	Captures CSR configuration (command, width, dummy count) and acknowledges pending CSR requests
S_WAIT_TRANS	Asserts <code>o_ax_write_valid</code> to accept write data from the AXI SCLK logic
S_IND_CMD	Sends an independent (command-only) transaction to the PSRAM

State	Description
S_CMD	Drives the command phase on QSPI pads
S_ADDR	Drives the address phase on QSPI pads
S_DUMMY	Drives the dummy phase (tri-state); cycle count scaled per mode
S_WRITE	Drives write data onto QSPI pads
S_READ	Samples read data from QSPI pads

7.2 Arbitration

The arbiter (`m_vlsi_arbiter`) performs round-robin selection between concurrent read and write requests in the SCLK domain:

```
o_arb_sel      = i_req_read  & (~i_req_write | arb_toggle); // 1 = read wins
o_arb_write_en = i_req_write & (~i_req_read  | ~arb_toggle); // 1 = write wins
```

`arb_toggle` flips after each granted operation, ensuring neither direction starves the other.

Request conditions:

```
req_write = ~awfifo_empty & ~wfifo_empty & (~aw_last | ~bfifo_full)
req_read  = ~arfifo_empty & ~rd_pending & ~rfifo_full
```

7.3 Write Flow

1. AXI AXFSM (AW channel) generates per-beat addresses and pushes them into AWFIFO.
2. AXI bus logic pushes write data beats into WFIFO.
3. SCLK logic pops AWFIFO and WFIFO when a write is granted by the arbiter and the FSM is ready.
4. The FSM executes: S_SET_UP → S_WAIT_TRANS → S_CMD → S_ADDR → S_DUMMY → S_WRITE.
5. BFIFO receives `{aw_id, bresp}` after the last write beat. `bresp = SLVERR (2'b10)` if the AW-channel burst length does not match the W-channel beat count; `OKAY (2'b00)` otherwise.

7.4 Read Flow

1. AXI AXFSM (AR channel) generates per-beat addresses and pushes them into ARFIFO.

2. SCLK logic pops ARFIFO when a read is granted and no read is pending.
3. The FSM executes: S_SET_UP → S_CMD → S_ADDR → S_DUMMY → S_READ.
4. Read data is pushed into RFIFO as {rd_id, 2'b00, rdata, rd_last}.
5. AXI bus logic pops RFIFO and drives the AXI R channel.

7.5 Independent Command

An independent command is a command-only transaction (no address or data phase). It is used for PSRAM control operations such as transitioning between SPI and QSPI modes.

Sequence: 1. Configure `independent.cmd`, `independent.mode`, and set `independent.req = 1`. 2. The FSM completes any in-progress AXI transaction before servicing the independent request. 3. FSM enters S_IND_CMD, sends the command on QSPI pads, then transitions the interface to `independent.mode`. 4. `independent.ack` asserts when the FSM acknowledges; both `req` and `ack` clear automatically.

7.6 XIP Mode

XIP (Execute-In-Place) is not controlled by a dedicated CSR bit. The FSM infers XIP behavior from the AXI burst length:

- Single-beat AXI transaction → non-XIP access (single read/write)
- Multi-beat AXI burst → interpreted as XIP streaming access

7.7 auto_data_wd

When `ctrl.auto_data_wd = 1`, the FSM overrides the `wd.data` configuration on every transaction entering S_WRITE or S_READ, setting the data width to `PARAM_DATA_WD - 1` automatically. This is useful when the PSRAM always transfers full-width words and manual width tracking is not required.

7.8 Write Response Error (SLVERR)

`BRESP = SLVERR (2'b10)` is generated when the burst length indicated on the AW channel does not match the actual number of W-channel beats received (`aw_last XOR w_last` mismatch at transaction time). This protects against malformed bursts.

VIII. Software Integration Guide

8.1 Initialization

1. Assert `i_qspi_rstn`, `i_axi_rstn`, `i_apb_rstn` (active low – hold low during init).
2. Release resets after clocks are stable.
3. Configure CSR registers via APB:
 - a. Write `wr_dummy.num` and `rd_dummy.num`.
 - b. Write `write.cmd` and `read.cmd`; set `req` bits and poll `ack` bits.
 - c. Write `wd.addr` and `wd.data`; set `wd.req` and poll `wd.ack`.
 - d. If starting in QSPI mode, issue an independent command:
 - Write `independent.cmd`, `independent.mode = 2'b10` (QSPI), `independent.req = 1`.
 - Poll `independent.ack` until asserted.
4. Write `ctrl.en = 1` to enable the IP.
5. The IP is now ready to accept AXI transactions.

8.2 Normal AXI Read/Write

After initialization, issue standard AXI4 read and write transactions. The IP handles burst address generation, CDC, arbitration, and QSPI sequencing transparently. No additional software intervention is required per transaction.

- AXI address maps directly to the PSRAM address.
- Burst types FIXED, INCR, and WRAP are all supported.
- Monitor `o_bresp` on the B channel; `SLVERR` indicates a burst-length mismatch.

8.3 Changing Read/Write Commands at Run Time

To replace the active read or write command (e.g., switching to a Read-ID instruction):

1. Write the new command value to `read.cmd` (or `write.cmd`).
2. Set `read.req = 1` (or `write.req = 1`).
3. Poll `read.ack` (or `write.ack`) until it asserts.
4. Issue the special AXI transaction.
5. Restore the standard command value and repeat steps 1–3 before resuming normal accesses.

If data or address width must also change, follow the same `req/ack` pattern for the `wd` register.

8.4 Independent (Command-Only) Transaction

1. Write `independent.cmd` with the PSRAM command opcode.
2. Write `independent.mode` with the target interface mode (00=SPI, 10=QSPI).
3. Set `independent.req = 1`.

4. Poll independent.ack until it asserts (indicates FSM accepted the request).
5. Both req and ack clear automatically; no explicit clear required.

Note: Any pending AXI transactions will complete before the independent command is serviced.

8.5 APB Access Latency

Every APB register access (read or write) is stalled until the 4-phase handshake across the PCLK/SCLK boundary completes. The minimum latency is approximately 2 SCLK cycles + 2 PCLK cycles (synchronizer stages), plus register setup time. Software must not assume zero-latency APB accesses; use a polling loop on PREADY or rely on the APB bus infrastructure to handle the stall.

IX. References

Component	Repository
AXI Controller Function	https://github.com/nguyenquanicd/AXI4-SRAM-CONTROLLER
CSR Interface (APB CSR Generator)	https://github.com/nguyenquanicd/APB-CSR-Generator
PSRAM Model (used for verification)	https://github.com/chipfoundry/EF_PSRAM_CTRL-1

X. Contact

This IP is developed and maintained by **VLSI Technology**. Designers: **Ethan, Link**
For questions, please contact the VLSI Technology page.